



科学计算与系统建模仿真平台MWORKS师资培训系列讲座

科学计算模块-Julia

© 苏州同元软控信息技术有限公司 2022

苏州同元软控技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

回顾

序号	主题	内容	时间
1	科学计算与系统建模仿真平台MWORKS介绍与使用	1、同元软控简介 2、新一代科学计算与系统建模仿真平台MWORKS 3、系统建模仿真环境-Sysplorer 4、科学计算环境-Syslab 5、高校数字化教学典型应用场景	2022年09月14日
2	MWORKS功能及其与MATLAB的对比	1、MWORKS功能与MATLAB功能的映射对比 2、应用流程对比 3、功能差异性对比(科学计算、系统建模)	2022年11月17日
3	系统建模仿真模块	1、Modelica语言发展历史及特点 2、Modelica语言生态 3、MWORKS.Sysplorer功能及运行过程 4、Modelica建模实例 5、Modelica语法概览	2022年12月29日

每次培训均有录屏，地址：<http://58.210.85.21:88/d/316b35bc68e04acc92a0/>

目录

1. Julia语言发展历史及特点

2. Julia语言生态

3. MWORKS.Syslab功能

4. Julia语法与其他语言区别

5. 如何构建高效的Julia代码

1.1 Julia语言发展历史

常用的计算机语言：

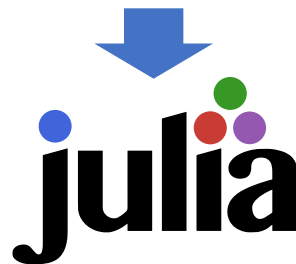
- MATLAB
- R
- JAVA
- Mathematica
- Python
- Rust
-

每一种语言都对某一项工作的一项特定需求非常完美，但是却无法胜任其它需求。

于是，使用什么语言都需要我们去权衡。

而我们很贪婪，我们还想要更多：

- 拥有C的速度和Ruby的动态性
- 有着真正和lisp一样的宏
- 像MATLAB一样类似于数学表达式
- 可以像Python一样作为通用编程语言
- 可以像R那样适用于统计分析
- 能像Perl那样自然地处理字符串
- 能像MATLAB一样给力地处理举证
- 能像shell一样作为胶水将各个程序粘合在一起
-



1.1 Julia语言发展历史



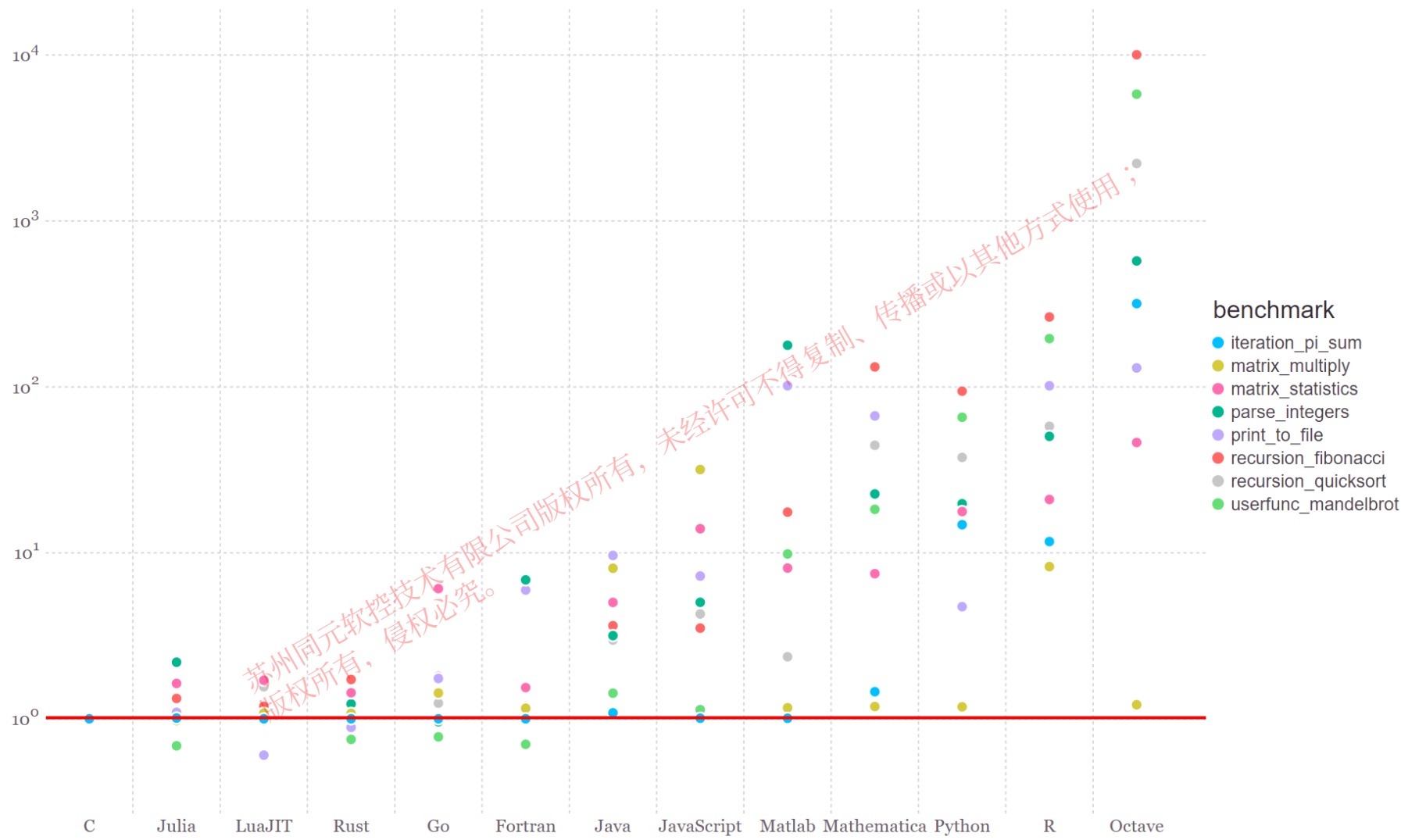
1.1 Julia语言发展历史

Position	Programming Language	Ratings
21	Julia	0.61%
22	Scratch	0.57%
23	SAS	0.56%
24	(Visual) FoxPro	0.52%
25	COBOL	0.52%
26	Rust	0.51%
27	Prolog	0.45%
28	Ada	0.43%
29	Lua	0.41%
30	Lisp	0.37%
31	PL/SQL	0.36%
32	Dart	0.34%
33	Scala	0.33%
34	Kotlin	0.31%
35	D	0.27%
36	PowerShell	0.21%
37	ABAP	0.21%
38	Awk	0.20%
39	LabVIEW	0.20%
40	TypeScript	0.20%
41	Groovy	0.18%
42	Erlang	0.15%
43	Haskell	0.15%
44	cg	0.15%
45	Transact-SQL	0.15%
46	Perl	0.14%

Julia在编程语言中排名：**21**，来自 TIOBE 2022年9月榜单



1.2 Julia语言特点



2018年

From <https://julialang.org/benchmarks/>

1.2 Julia语言特点

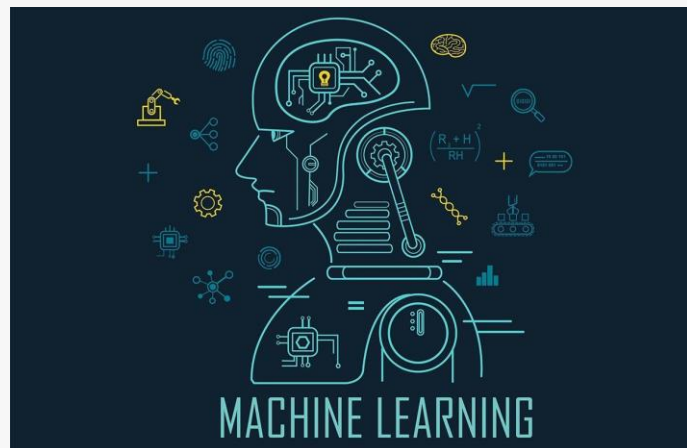
Julia 语言是一门**高级、通用、动态的**程序设计语言，适合用于与**科学计算**相关的高性能计算领域。

特点

- 数学形式表达
- 动态编译型语言
- 函数式编程+多重派发
- 广播

应用领域

- 统计学与数据科学
- 工程学
- 深度学习
- 机器学习
- 计算机科学
- 物理学
- 数学
- 人工智能
- 信号与图像处理



1.2 Julia语言特点-数学形式表达

```
julia> A = [1 2 3;4 5 6]
2×3 Matrix{Int64}:
 1  2  3
 4  5  6
```

```
julia> b = A[1,1]
1
```

```
julia> B = [1 2 3
            4 5 6]
2×3 Matrix{Int64}:
 1  2  3
 4  5  6
```

```
julia> c = B[2,1]
4
```

数组索引从1开始

```
julia> α = 0.5; β = 0.2; γ = 0.1;
```

```
julia> Rx = [1 0 0
            0 cos(α) -sin(α)
            0 sin(α) cos(α)]
```

```
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 0.0  0.877583 -0.479426
 0.0  0.479426  0.877583
```

```
julia> Ry = [cos(β) 0 sin(β)
            0 1 0
            -sin(β) 0 cos(β)]
```

```
3×3 Matrix{Float64}:
 0.980067  0.0  0.198669
 0.0       1.0  0.0
-0.198669  0.0  0.980067
```

公式中的特殊字符也可直接定义

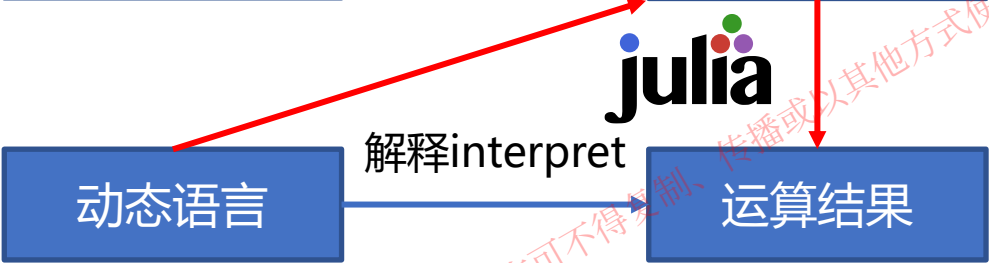


1.2 Julia语言特点-动态编译型语言

静态语言(C/C++)
建模效率低，执行效率高

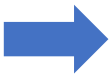


动态语言(Python/MATLAB)
建模效率高，执行效率低



一般方式:

1. 在动态语言中进行快速建模和概念验证
2. 发现性能太差
3. 性能瓶颈有关的代码切换到静态语言中进行重新实现
4. 再写从低级语言到高级语言的接口代码



"两语言问题":

- 对于大部分科研人员而言，在静态语言(C/C++)中实现太慢且太难
- 直接从动态语言翻译到静态语言不一定得到理想的加速
- 跨语言调用的胶水代码有时候甚至比核心代码还要多
- 跨语言调用意味着黑盒模型：一些优化策略无法使用

1.2 Julia语言特点-动态编译型语言

静态语言(C/C++)
建模效率低，执行效率高



动态语言(Python/MATLAB)
建模效率高，执行效率低




能编译时编译，不能编译的时候解释



第一次运行时较慢

解决方式

- 1. 镜像文件
- 2. Julia precompile, Julia 1.9将大幅提升!



1.2 Julia语言特点-动态编译型语言

```
In [8]: %timeit my_sum(B)
215 ms ± 1.19 ms per loop (mean ± std. dev.
```

Python语言构造函数

```
In [1]: import numpy as np
In [2]: A = np.random.rand(10000, 10000)
In [3]: %timeit np.sum(A)
39 ms ± 281 µs per loop (mean ± std. dev. of 7 runs,
In [4]: B = np.random.rand(1000, 1000)
In [5]: %timeit np.sum(B)
217 µs ± 3.58 µs per loop (mean ± std. dev. of 7 runs
```

Python直接调用numpy, numpy底层为C

```
julia> A = rand(10000, 10000);
julia> my_sum(A) ≈ sum(A)
true
julia> @btime my_sum($A);
47.302 ms (0 allocations: 0 bytes)
julia> B = rand(1000, 1000);
julia> @btime my_sum($B);
117.991 µs (0 allocations: 0 bytes)
```

Julia语言构造函数

1.2 Julia语言特点-函数式编程+多重派发

函数式编程：将尽可能多的简短的函数组合一个完整的功能

```
julia> A = rand(1000, 1000);  
  
julia> norm(A) ≈ sqrt(mapreduce(abs2, +, A))  
true  
  
julia> norm(A) ≈ sqrt(sum(abs2, A))  
true
```

多重派发：函数 (Function) 由多个方法 (Method) 共同定义，根据具体的数据类型决定实际调用的方法

```
julia> add(x::Number, y::Number) = x + y  
add (generic function with 1 method)  
  
julia> add(x::String, y::String) = "$x$y"  
add (generic function with 2 methods)  
  
julia> add(1, 2)  
3  
  
julia> add("hello", " world")  
"hello world"
```

苏州同元软控技术有限公司版权所有，未经授权不得转载，侵权必究。

1.2 Julia语言特点-函数式编程+多重派发

单重派发 VS 多重派发

Python 单重派发：根据第一个输入的类型决定具体调用的方法

手动派发

```
class Point2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def dist(self, p):
        return math.sqrt((self.x - p.x)**2 + (self.y - p.y)**2)
```

```
def dist(self, p):
    if isinstance(p, Point2D):
        return math.sqrt((self.x - p.x)**2 + (self.y - p.y)**2)
    elif isinstance(p, int):
        return self.dist(Point2D(p, p))
```

Julia 多重派发：根据所有输入的类型决定具体调用的方法

```
julia> dist(p::Point2D, q::Point2D) = sqrt((p.x-q.x)^2 + (p.y-q.y)^2)
dist (generic function with 1 method)

julia> dist(p::Point2D, q::Number) = dist(p, Point2D(q, q))
dist (generic function with 2 methods)

julia> dist(Point2D(3, 4), Point2D(0, 0))
5.0

julia> dist(Point2D(3, 4), 0)
5.0
```

单重派发的问题：

- 代码过于冗长和复杂
- 扩展问题：C想基于B扩展A，A不愿意
- 所有权导致的重复造轮子

1.2 Julia语言特点-函数式编程+多重派发

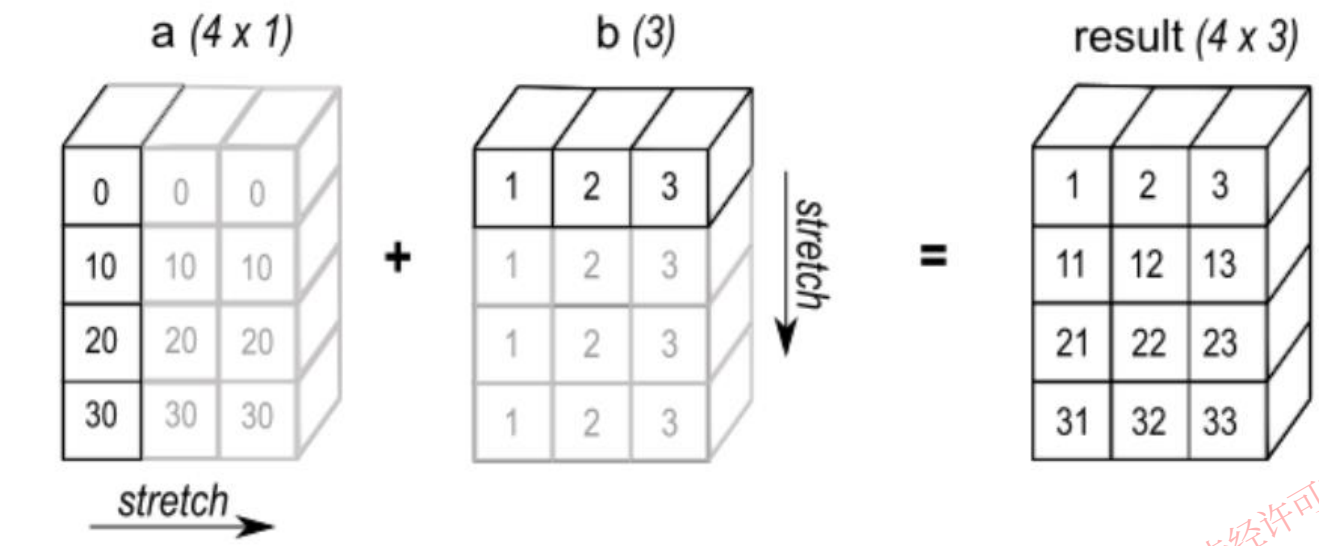
Language	Average # methods (DR)	Choice ratio (CR)	Degree of specialization (DoS)
Cecil ^[2]	2.33	63.30	1.06
Common Lisp (CMU) ^[2]	2.03	6.34	1.17
Common Lisp (McCLIM) ^[2]	2.32	15.43	1.17
Common Lisp (Steel Bank) ^[2]	2.37	26.57	1.11
Diesel ^[2]	2.07	31.65	0.71
Dylan (Gwydion) ^[2]	1.74	18.27	2.14
Dylan (OpenDylan) ^[2]	2.51	43.84	1.23
Julia ^[3]	5.86	51.44	1.54
Julia (operators only) ^[3]	28.13	78.06	2.01
MultiJava ^[2]	1.50	8.92	1.02
Nice ^[2]	1.36	3.46	0.33

多重派发作为核心编程风格

From https://en.wikipedia.org/wiki/Multiple_dispatch



1.2 Julia语言特点-广播



```
julia> a = [0;10;20;30]
4-element Vector{Int64}:
 0
10
20
30
```

```
julia> b = [1 2 3]
1x3 Matrix{Int64}:
 1  2  3
```

```
julia> a .+ b
4x3 Matrix{Int64}:
 1  2  3
11 12 13
21 22 23
31 32 33
```

```
julia> A = magic(3)
3x3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2

julia> A .+ 1
3x3 Matrix{Int64}:
 9  2  7
 4  6  8
 5 10  3
```

```
julia> A .+ [1,2,3]
3x3 Matrix{Int64}:
 9  2  7
 5  7  9
 7 12  5
```

```
julia> sqrt.(A)
3x3 Matrix{Float64}:
 2.82843  1.0  2.44949
 1.73205  2.23607  2.64575
 2.0      3.0  1.41421
```

未经许可不得复制、传播或以其他方式使用；
苏州同元软控技术有限公司版权所有，侵权必究。



目录

1. Julia语言发展历史及特点

2. Julia语言生态

3. MWORKS.Syslab功能

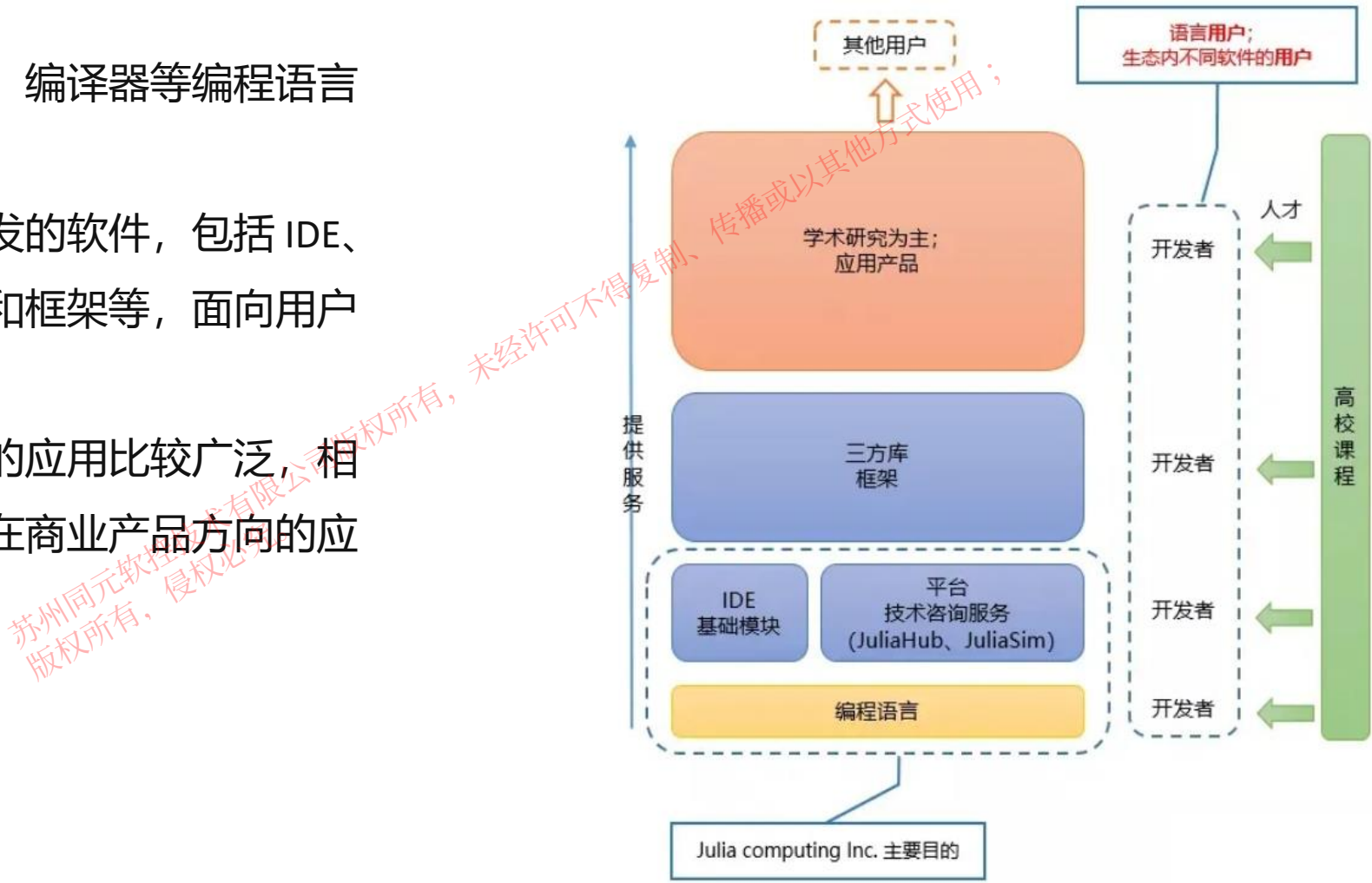
4. Julia语法与其他语言区别

5. 如何构建高效的Julia代码

2 Julia语言生态

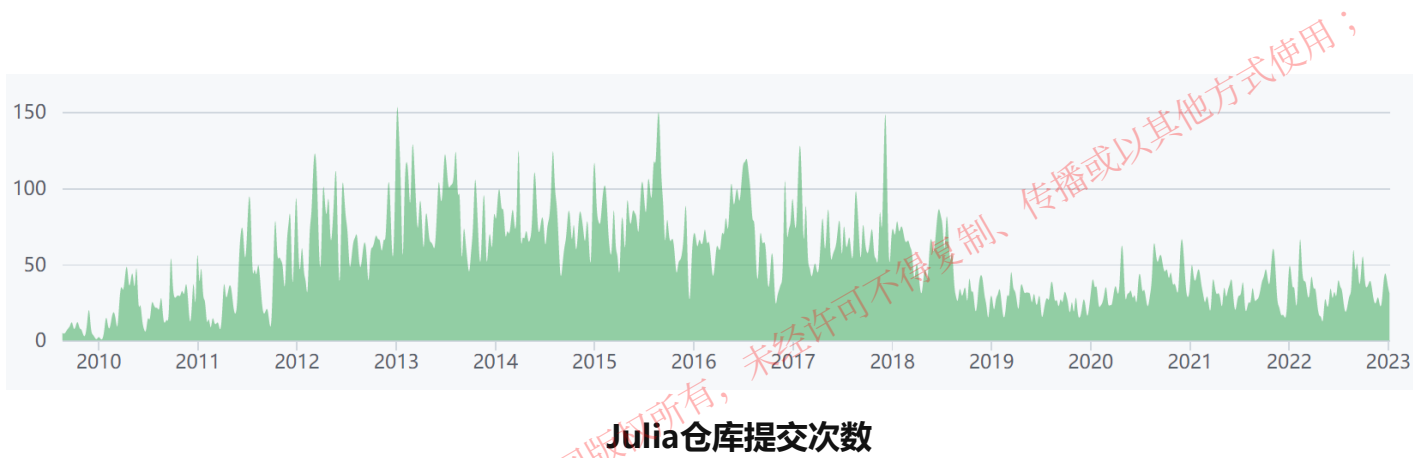
Julia 语言的生态系统还在飞速成长的阶段，当前可以大致分为**语言项目**、**工具**、**应用**三部分来看：

- **语言项目**：包括语言设计、编译器等编程语言的实现项目。
- **工具**：围绕语言的使用研发的软件，包括 IDE、基础模块、平台、三方库和框架等，面向用户也为开发人员。
- **应用**：目前在学术研究中的应用比较广泛，相关的库、工具比较完善，在商业产品方向的应用相对较少。



2 Julia语言生态-语言项目

Julia 语言项目在 GitHub 上开源，语言仓库主要开发迭代是 2012 年至 2018 年。



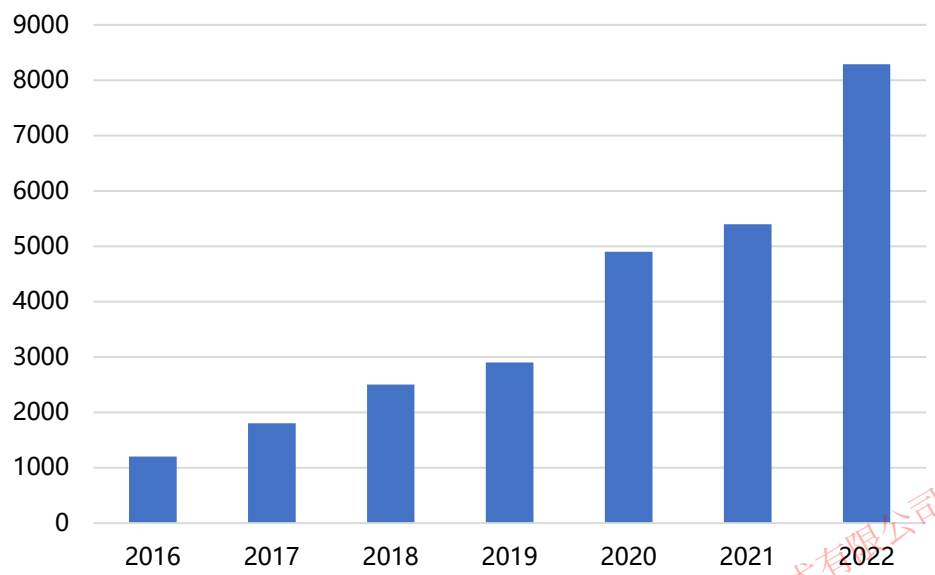
当前开启的 Issues 超过 3000 个，Issues 关联最多的标签是 **Performance**、**doc** 和 **bug**。

其中 **Performance** 标签的描述是 “**Must go faster**”，

Issues 主要是关于使用 Julia 编写的代码优化以达到更快的速度，可见选择 Julia 的用户对于性能的重视。

2 Julia语言生态-工具

注册包数量



截至2022年12月，共计8287个工具箱

科学计算：

- DifferentialEquations
- LightGraphs
-

人工智能：

- Flux
- Knet
-

并行计算：

- CUDA
- ParallelAccelerator
-

图形与图像处理：

- Plots
- Images
-

优化算法：

- Optim
- Distributions
-

生物学：

- Bio
- PhyloNetworks
-

Julia拥有丰富的开源生态资源，更多行业工具箱
详见：<https://juliapackages.com/>



2 Julia语言生态-工具

Julia已有的工具软件生态主要可以支持通用应用、可视化、数据科学、机器学习、科学计算和并行计算等实现：

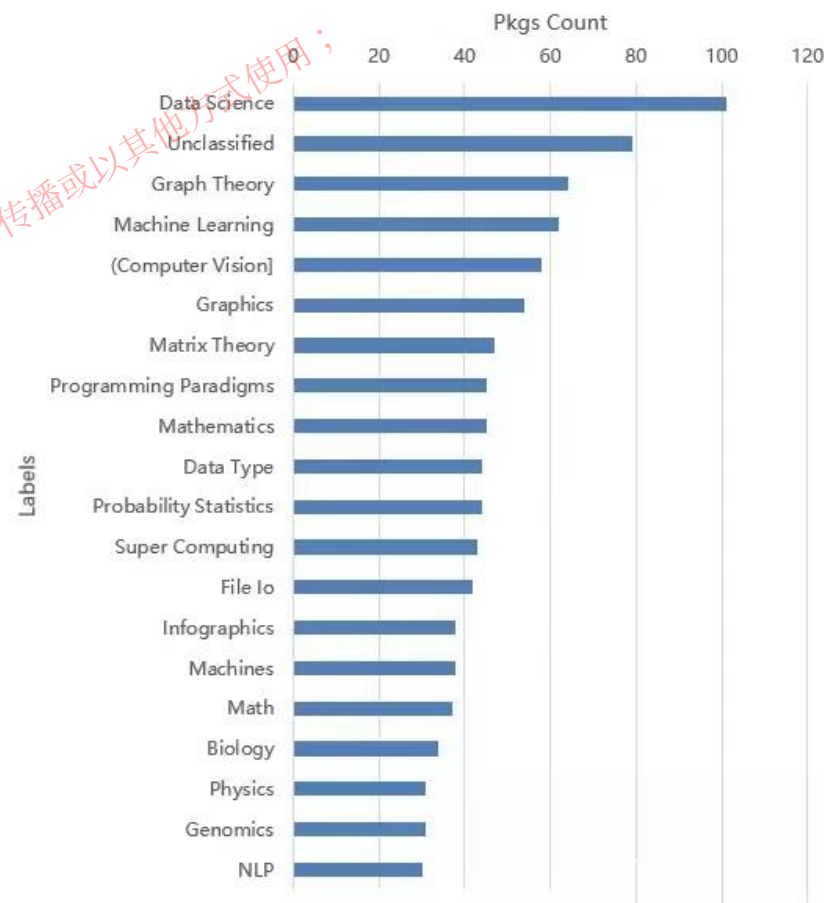
苏州同元软控技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

2 Julia语言生态-工具

Julia生态最丰富的领域，如最先进的微分工具系统（DifferentialEquations.jl）、优化工具（JuMP.jl and Optim.jl）、迭代线性求解器（IterativeSolvers.jl）、快速傅里叶变换（AbstractFFTs.jl）等等，在其他特定领域也有不同的库，如生物学（BioJulia）、运筹学（JuMP Dev）、图像处理（JuliaImages）等。

注册的库已超过 8000 个，在 Julia Observer 上可以看到 **Data Science、Graph Theory 和 Machine Learning** 三个标签相关库的数量最多。

Julia 还提供了 Juno、VS Code、Jupyter 等主流编辑器和调试、代码分析等工具的支持。



Julia Observer各标签相关的库数量



2 Julia语言生态-工具

Julia Computing 公司

该公司成立于 2015 年，其成立最重要的目的就是为了发展 Julia 语言。其旗舰产品 JuliaHub 是一个 SaaS (Software as a Service) 平台，允许用户直接在平台上使用 Julia 开发、部署应用，并扩展上千个节点。该平台同时提供了 Pumas、JuliaSim、JuliaSPICE 等专业领域的前沿产品，可用于药物、多物理场和电子电路的建模和仿真模拟。

其中和 Pumas-AI 在制药领域的合作，给各大疫苗包括新冠疫苗研发中提供了较大的助力，使得 Julia Computing 成功走入大众视野。



CODE

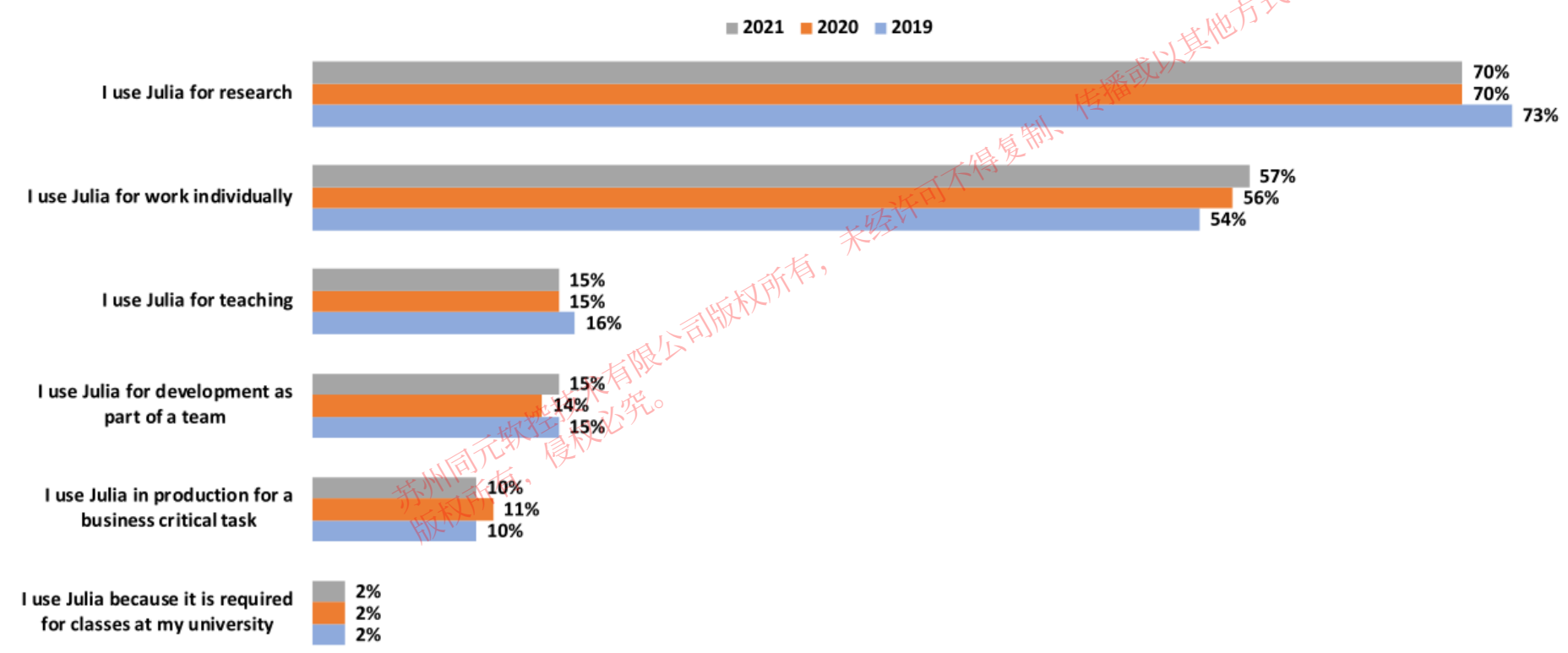
Performing Non-Compartmental Analysis with Julia and Pumas AI

by Nikolay Manchev December 4, 2020 8 min read



2 Julia语言生态-应用

- Julia 在学术研究中应用广泛，官网Julia 的论文《Julia: A Fresh Approach to Numerical Computing》已被引用超过 2500 次。
- 在 2021 年 Julia 开发者调研中显示有 70% 用户使用 Julia 进行学术研究。



2 Julia语言生态-应用

Julia语言已深度应用于新冠传播分析和疫苗研制，MIT已发布相关公开课，300名学生参加。

MITOPENCOURSEWARE

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

GIVE NOW

ABOUT OCW

HELP & FAQs

CONTACT US

18.S190 | Spring 2020 | Undergraduate

Introduction To Computational Thinking With Julia, With Applications To Modeling The COVID-19 Pandemic

Syllabus

Course Materials

Assignments

COURSE DESCRIPTION

This half-semester course introduces computational thinking through applications of data science, artificial intelligence, and mathematical models using the Julia programming language. This Spring 2020 version is a fast-tracked curriculum adaptation to focus on applications to COVID-19 responses.

See the MIT News article [Computational Thinking Class Enables Students to Engage in Covid-19 Response](#)

Show less

COURSE INFO

Instructors

[Prof. Alan Edelman](#)
[Prof. David P. Sanders](#)

Departments

[Mathematics](#)
[Electrical Engineering and Computer Science](#)

Topics

Engineering

Computer Science

[Algorithms and Data Structures](#)
[Computer Design and Engineering](#)
[Computer Networks](#)

Mathematics

LEARNING RESOURCE TYPES

Lecture Videos

Problem Sets

OUR NEW MODELS
OUTLINE A FEW
POSSIBLE SCENARIOS.

#1 IS THE
BEST CASE
SCENARIO.

①

BAD STUFF ↑

TIME →

It's important to consider a variety of possible scenarios when looking at data to potentially predict the future. (Courtesy of [Randall Munroe](#). License: CC-BY-NC.)

Download Course

详见：<https://news.mit.edu/2020/computational-thinking-class-enables-students-engage-covid-19-response-0407>

25

Copyright © 2023 苏州同元软控信息技术有限公司
All rights reserved

2 Julia语言生态-应用

全球超过 **1500** 所高校已经在使用 Julia 并教授相关知识，包括麻省理工、斯坦福大学、加州大学伯克利分校等世界一流学府。

目前全球已经有超过 **10000** 家公司使用 Julia 语言，其中包括谷歌、微软、英特尔、亚马逊、IBM、辉瑞、葛兰史素克、苹果、迪士尼以及NASA等重量级用户。



Apple



目录

1. Julia语言发展历史及特点

2. Julia语言生态

3. MWORKS.Syslab功能

4. Julia语法与其他语言区别

5. 如何构建高效的Julia代码

3.1 MWORKS.Syslab功能概览

MWORKS.Syslab基于科学计算高性能动态高级程序设计语言提供完整的交互式编程环境的完备功能，支持通用编程、科学计算、数据科学、机器学习、信号处理、通讯仿真、并行计算等功能。

主要功能:

交互式编程环境: Syslab通过资源管理器、代码编辑器、命令行窗口、工作空间、窗口管理等功能，提供了功能完备、强大的交互式编程、调试与运行环境

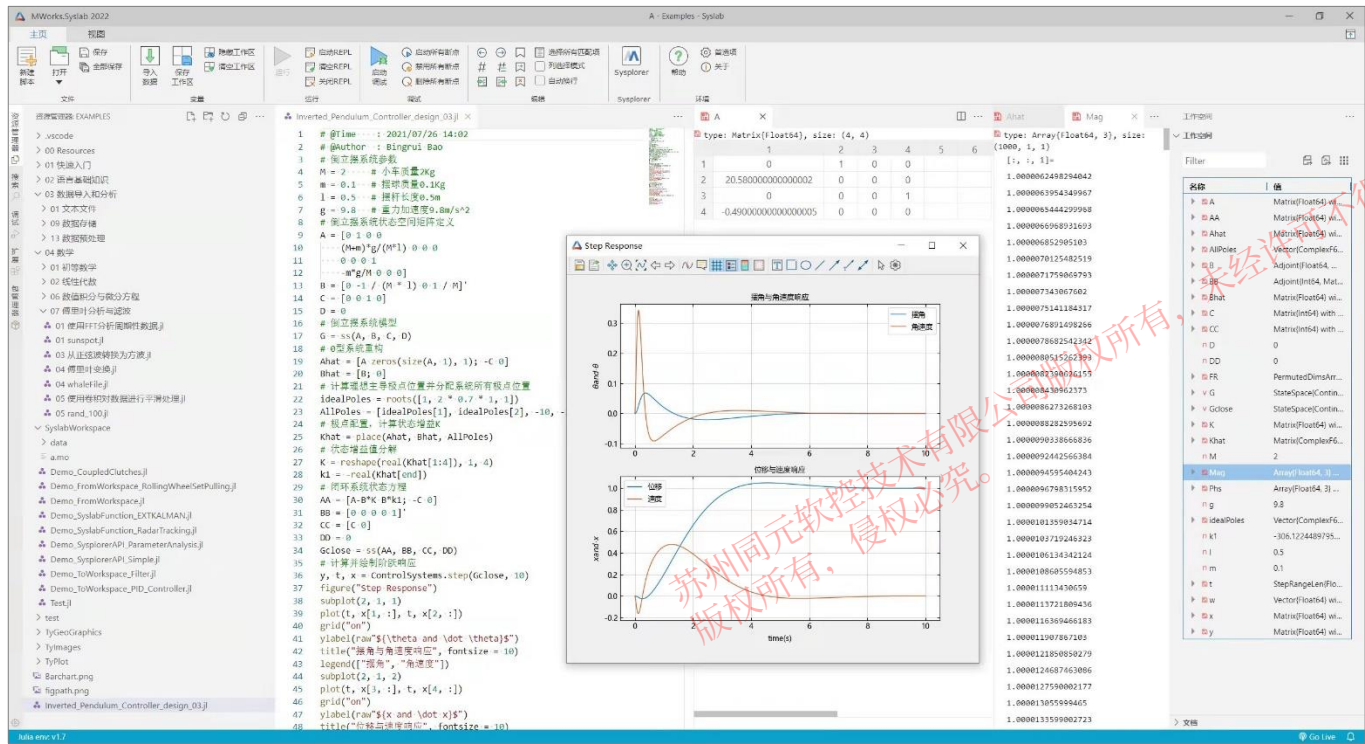
科学计算函数库: 提供数学、线性代数、矩阵与数组运算、插值、数值积分与微分方程、傅里叶变换与滤波、符号计算、曲线拟合、信号处理、通信等丰富的高质量、高性能科学计算函数

计算可视化图形: 内置大量易用的二维和三维绘图函数，支持数据可视化与图形界面交互，为数据分析及可视化提供工具支撑

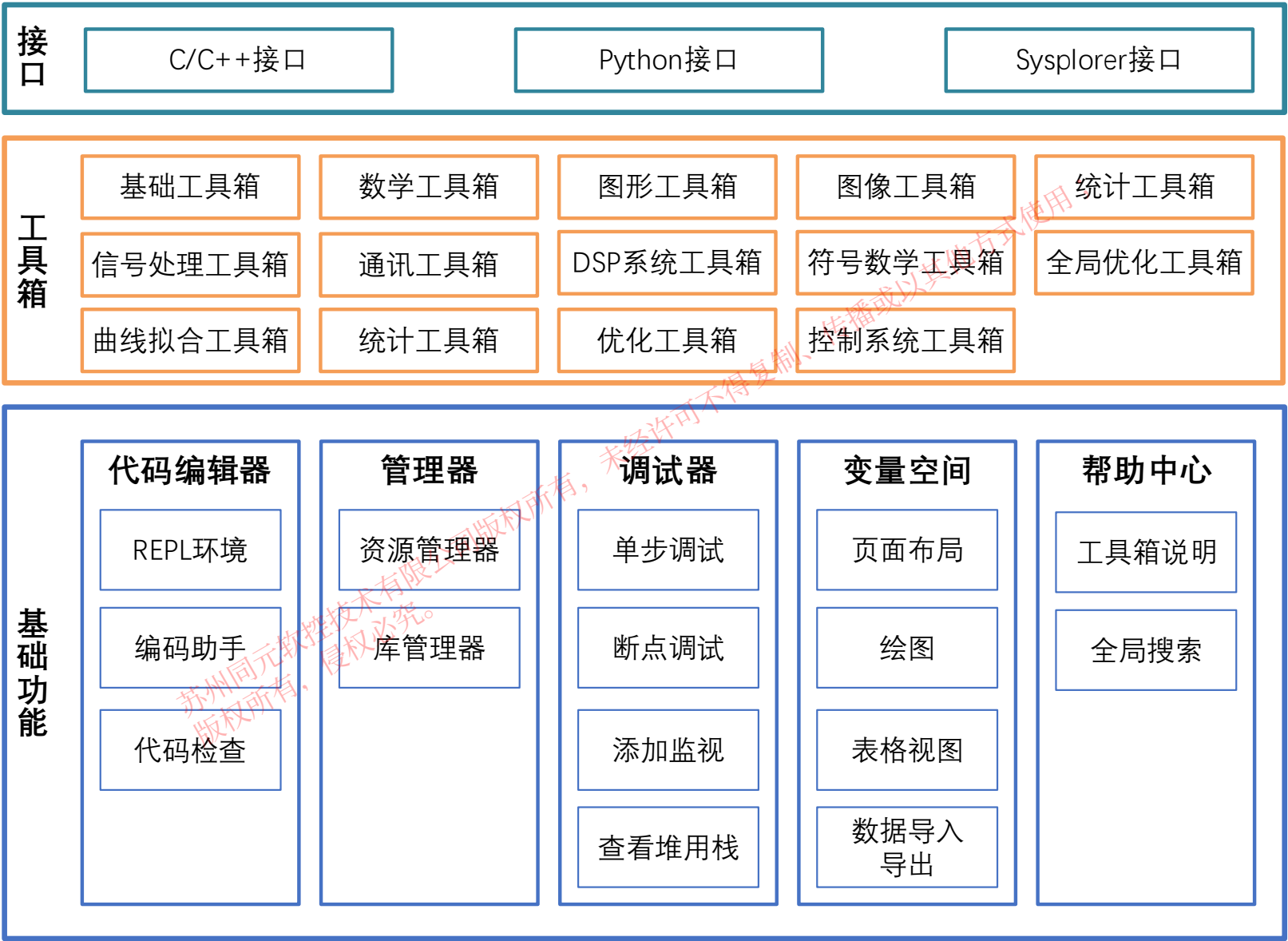
库开发与管理: 支持函数库的注册管理、依赖管理、加载卸载、版本切换，同时提供函数库开发规范，以支持用户自定义函数库的开发与测试

与系统建模环境深度融合: Syslab与系统建模环境Sysplorer之间实现了双向深度融合，优势互补，形成新一代科学计算与系统建模仿真平台

完善的中文帮助系统: 提供完整易用的中文帮助系统，包含深入全面的帮助主题和大量示例



3.1 MWORKS.Syslab功能概览



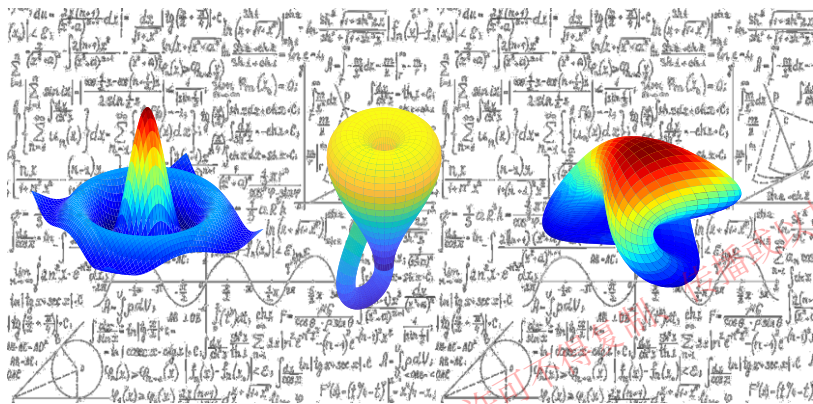
3.1 MWORKS.Syslab功能概览



3.2 工具箱

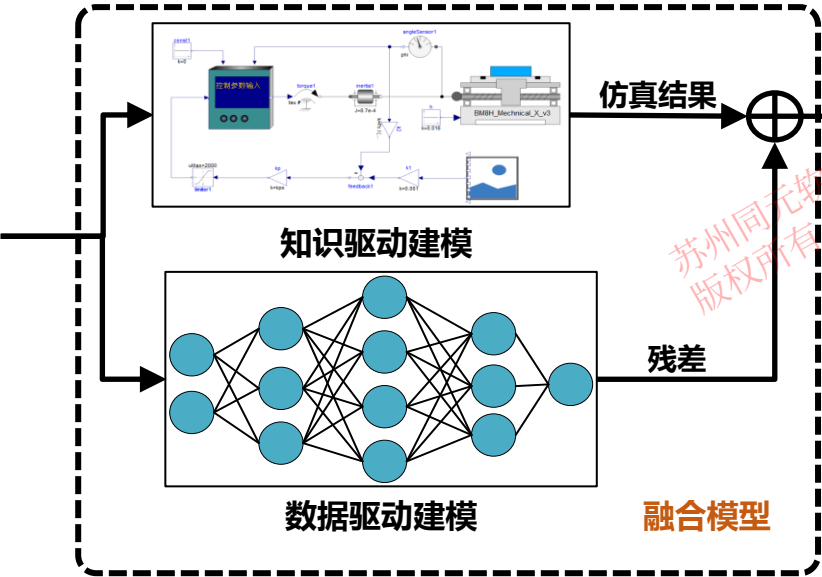
数学计算

数值计算、符号计算



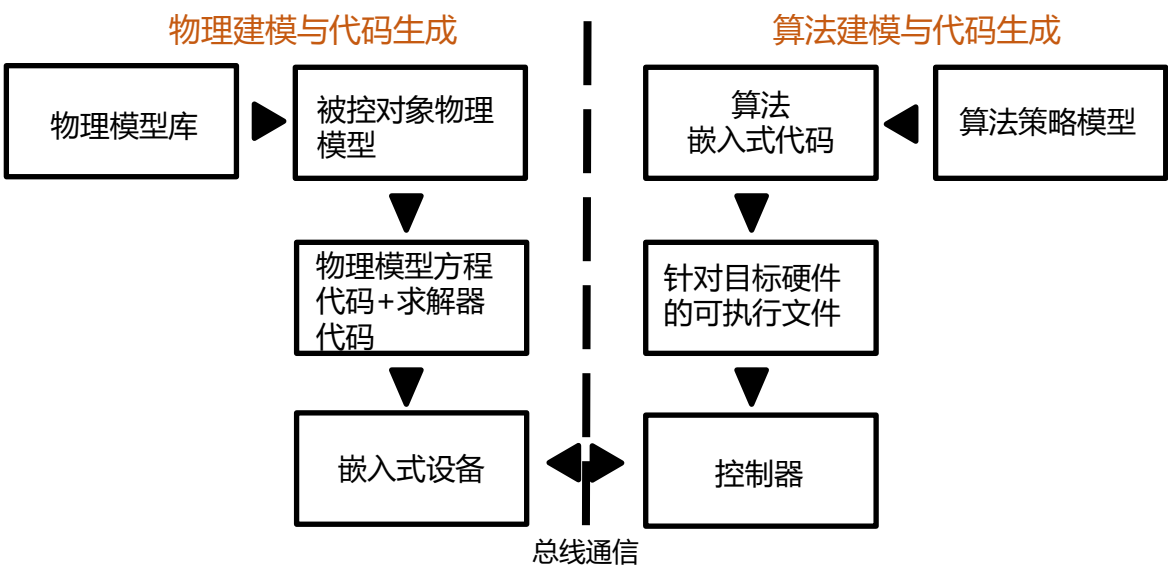
数据建模

数据处理、分析、建模、可视化



算法开发

控制、通信、AI



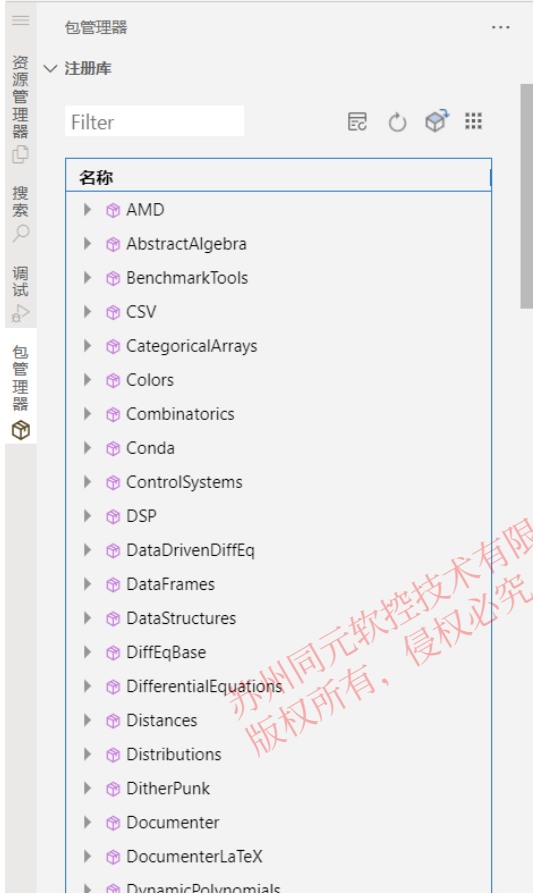
3.2 工具箱-TY工具箱

工具箱	内容
基础工具箱	• 输入命令 • 数组 • 数据类型 • 集合容器 • 初等运算和初等函数 • 流程控制 • 环境和设置 • 编程 • 数据导入和分析
数学工具箱	• 初等数学 • 线性代数 • 随机数生成 • 插值 • 数值积分和微分方程 • 傅里叶分析和滤波 • 稀疏矩阵
图形工具箱	• 二维图和三维图 • 格式和注释 • 打印和保存 • 图形对象
图像工具箱	• 显示图像 • 读取、写入和修改图像 • 转换图像类型 • 修改图像颜色
优化工具箱	• 非线性优化 • 非线性方程组 • 基于求解器的优化问题设置 • 线性规划和混合整数线性规划 • 最小二乘 • 二次规划与锥规划
全局优化工具箱	• 遗传算法 • 粒子群优化算法 • 模拟退火算法 • 方向搜索算法 • 多目标优化算法
统计工具箱	• 描述性统计量和可视化 • 概率分布 • 假设检验
深度学习工具箱	• 函数逼近、聚类和控制 • 深度学习自定义训练循环 • 图像深度学习 • 使用时间序列和序列数据进行深度学习

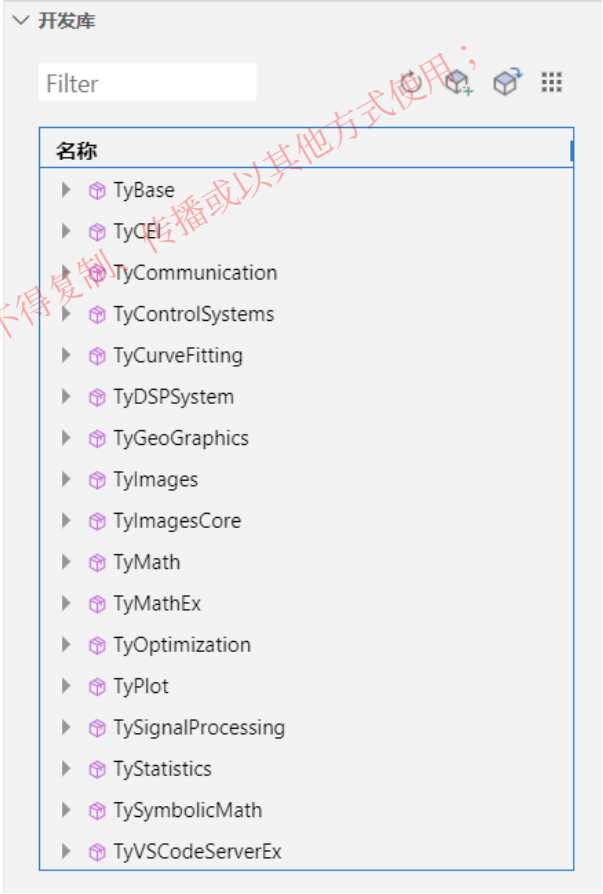
工具箱	内容
曲线拟合工具箱	• 预备教程 • 线性与非线性回归 • 插值 • 平滑 • 拟合后处理样条
信号处理工具箱	• 信号分析和可视化 • 信号生成及预处理 • 测量和特征提取 • 变换、相关性和建模 • 数字和模拟滤波器 • 频谱分析 • 信号的机器学习和深度学习延伸
通信工具箱	• 物理层组件 • 射频元件建模 • 传播和信道模型 • 测试和测量
DSP系统工具箱	• 信号产生、处理和分析 • 滤波器设计与分析 • 滤波器实现 • 变换与谱分析 • 统计学与线性代数
控制系统工具箱	• 系统创建 • 系统转换 • 系统互联 • 系统简化 • 系统特性 • 时域分析 • 频域分析 • 系统综合 • 矩阵运算 • 实用函数
地理图工具箱	• 绘制地理数据 • 自定义地理坐标区
符号数学工具箱	• 符号数学基础 • 符号数学
机器学习工具箱	• 分类 • 聚类 • 回归 • 工业统计 • 降维和特征提取

3.2 工具箱-包管理器

在Syslab中可以通过包管理器，管理注册库与开发库，点击刷新即可查看注册库和开发库。



已安装的第三方库



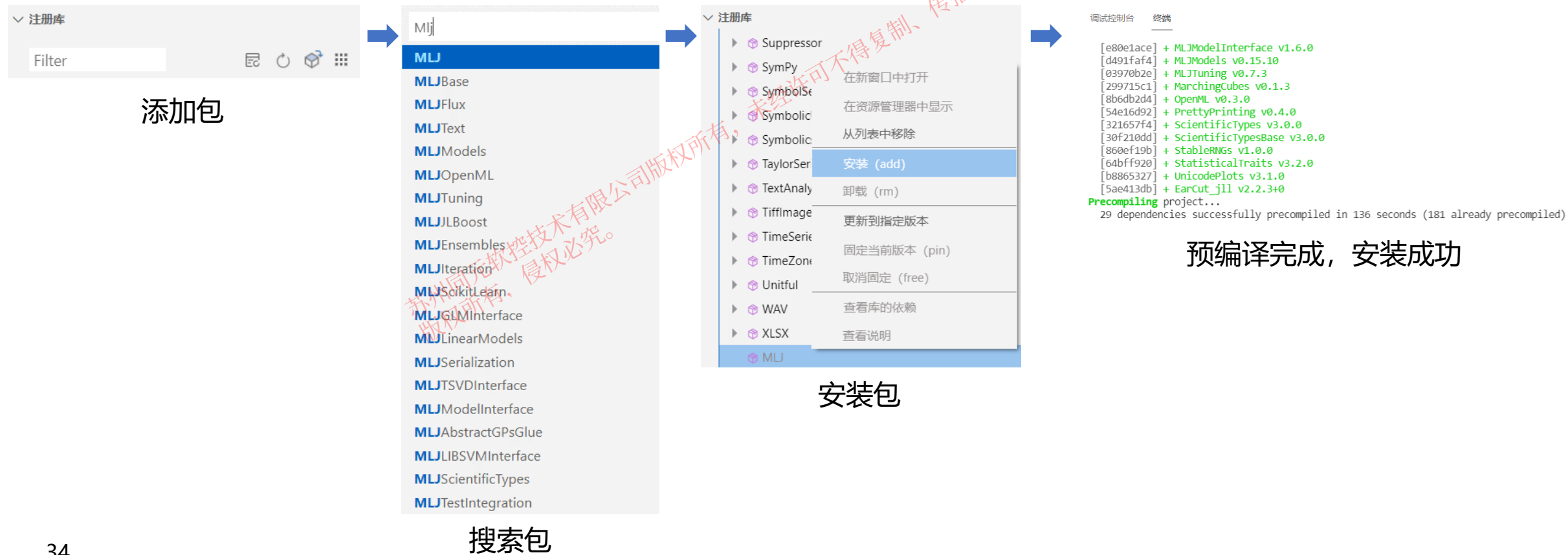
同元官方库

3.2 工具箱-包安装

方式1：可视化操作

操作步骤：

- 1. 注册库中点击“添加包”
- 2. 搜索第三方库，选择对应库，自动添加至注册库列表
- 3. 在注册库列表中右击，选择“安装”



3.2 工具箱-包安装

方式2：文本操作

操作步骤：

- 1. 在REPL中输入] 进入pkg模式(退出pkg模式可使用Backspace或Ctrl + C)
- 2. 在pkg模式中输入： add 第三方库名

(@v1.7) pkg> add MLJ



```
调试控制台  终端
[e80e1ace] + MLJModelInterface v1.0.0
[d491faf4] + MLJModels v0.15.10
[03970b2e] + MLJTuning v0.7.3
[299715c1] + MarchingCubes v0.1.3
[8b6db2d4] + OpenML v0.3.0
[54e16d92] + PrettyPrinting v0.4.0
[321657f4] + ScientificTypes v3.0.0
[30f210dd] + ScientificTypesBase v3.0.0
[860ef19b] + StablrNGs v1.0.0
[640ff920] + StatisticalTraits v3.2.0
[b8865327] + UnicodePlots v3.1.0
[5ae413db] + EarCut_jll v2.2.3+0
Precompiling project...
29 dependencies successfully precompiled in 136 seconds (181 already precompiled)
```

预编译完成，安装成功

方式1与方式2等价



3.2 工具箱-包安装

注意事项

1. 库下载慢，可在首选项中设置使用国内镜像源。



3. 可在pkg模式下键入“st”查看所有库的版本号

```
(@v1.7) pkg> st
Status `C:\Users\Public\TongYuan\.julia\environments\v1.7\Project.toml`
[14f7f29c] AMD v0.5.0
[c3fe647b] AbstractAlgebra v0.26.2
[4fba245c] ArrayInterface v6.0.23
[fbb218c0] BSON v0.3.5
[6e4b80f9] BenchmarkTools v1.3.1
[336ed68f] CSV v0.10.4
[052768ef] CUDA v3.12.0
[324d7699] CategoricalArrays v0.10.6
[5ae59095] Colors v0.12.8
[861a8166] Combinatorics v1.0.2
```

2. 预编译出错时，可能是依赖的某些第三方库版本不匹配，需用手动安装依赖库。

操作步骤：

- 1. 首选项中，关闭系统映像文件
- 2. 安装第三方依赖库 (add DiffEqBase@6.100)
- 3. 安装第三方库 (Flux, Image, ImageView等)
- 4. 构建映像
- 5. 首选项中，勾选系统映像文件

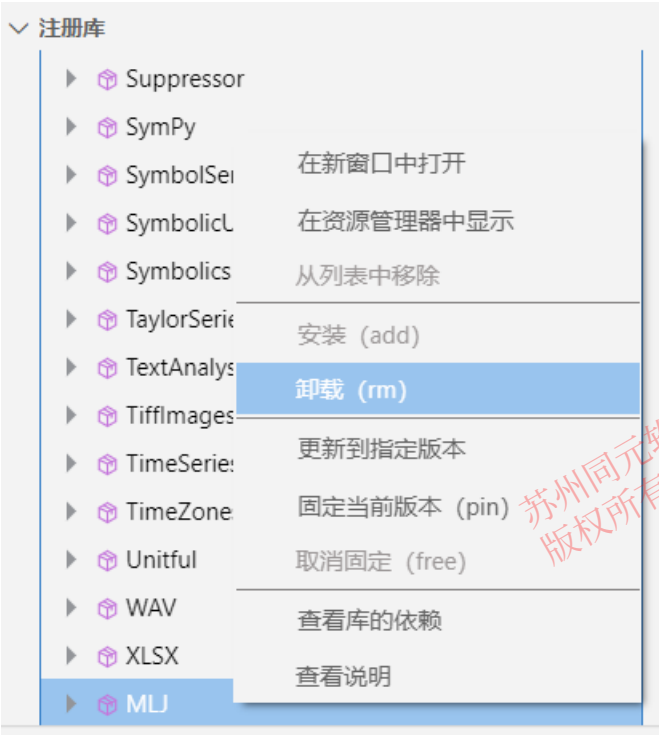


3.2 工具箱-包卸载

方式1：可视化操作

操作步骤：

1. 注册库中右击包，选择卸载



方式2：文本操作

操作步骤：

1. 在REPL中输入] 进入pkg模式(退出pkg模式可使用 Backspace或Ctrl + C)
2. 在pkg模式中输入： add 第三方库名

```
(@v1.7) pkg> remove MLJ
```

3.2 工具箱-包使用

在Julia中通过using使用已正确安装的第三方库，可直接调用第三方库中的函数。

```
using Flux, MLDatasets, Statistics
using Flux: onehotbatch, onecold, logitcrossentropy, Chain, params
using MLDatasets: MNIST
using Base.Iterators: partition
using Printf, BSON
using Parameters: @with_kw
using CUDA
```

```
model = build_model(args) #直接调用Flux库中的函数
```

```
julia> flatten
ERROR: UndefVarError: flatten not defined
#当多个库中的函数产生冲突时，需要使用库名+函数名的方式
julia> Flux.flatten
flatten (generic function with 1 method)
```

Flux和Base.Iterators库中均有flatten函数

3.2 工具箱-包使用

在Julia中通过using使用已正确安装的第三方库，可直接调用第三方库中的函数。

```
using Flux, MLDatasets, Statistics
using Flux: onehotbatch, onecold, logitcrossentropy, Chain, params
using MLDatasets: MNIST
using Base.Iterators: partition
using Printf, BSON
using Parameters: @with_kw
using CUDA
```

```
model = build_model(args) #直接调用Flux库中的函数
```

```
julia> flatten
ERROR: UndefVarError: flatten not defined
#当多个库中的函数产生冲突时，需要使用库名+函数名的方式
julia> Flux.flatten
flatten (generic function with 1 method)
```

Flux和Base.Iterators库中均有flatten函数

3.2 工具箱-实例

IIR低通滤波器设计

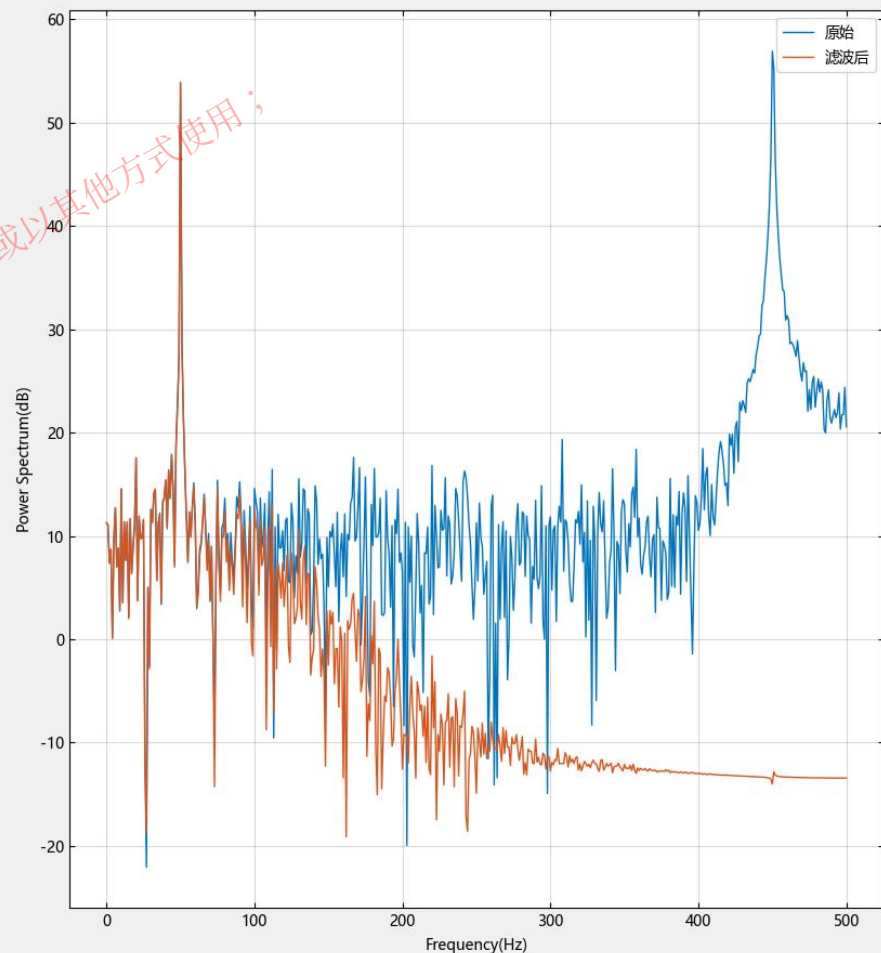
目标：设计双线性Z变化低通数字滤波器，分析其特点并验证性能

实现思路：

1. 确定滤波器性能指标
2. 预修正，将数字滤波器设计指标转换成模拟滤波器设计指标
3. 设计模拟低通滤波器
4. 使用双线性变换法将模拟低通滤波器转换成数字低通滤波器
5. 对滤波器进行频率响应分析
6. 生成不同频率信号，验证滤波器性能

相关函数：

1. buttord-巴特沃斯滤波器阶数和截止频率
2. buttap-巴特沃斯滤波器原型
3. lp2lp-更改低通模拟滤波器的截止频率
4. butter-巴特沃斯滤波器设计
5. bilinear-模数滤波器转换的双线性变换方法
6. freqz-离散时间滤波器的频率响应



3.2 工具箱-实例

基于深度学习的手写汉字识别系统的设计与实现

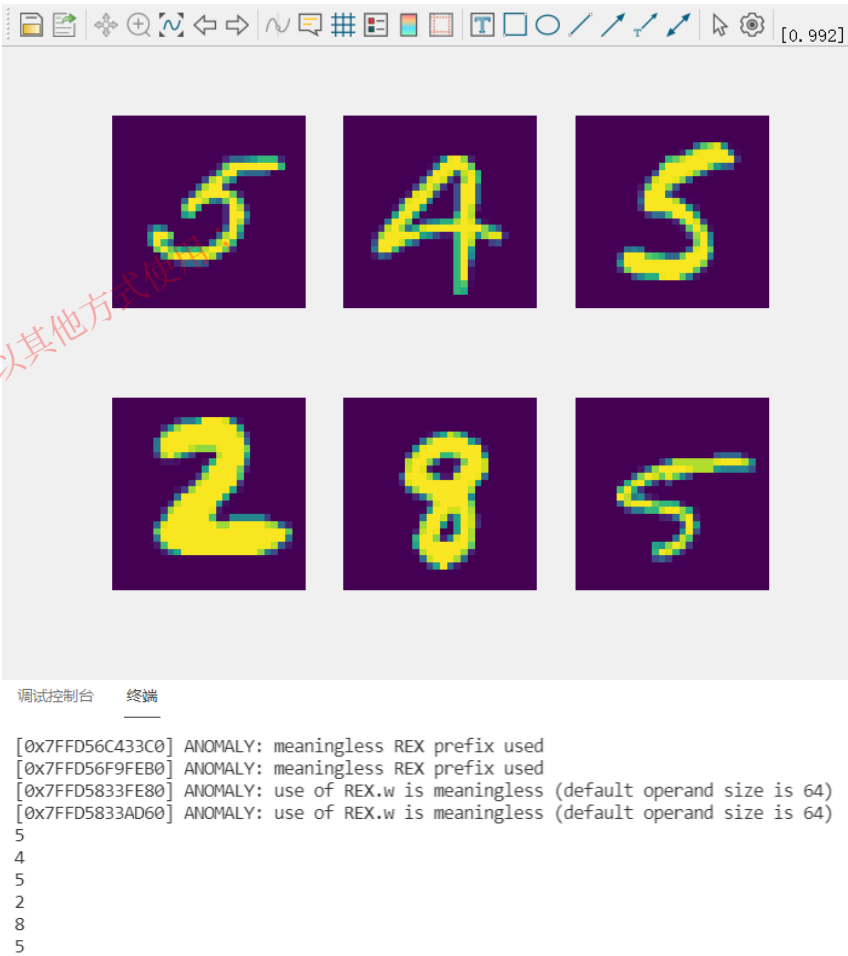
目标：使用卷积神经网络算法，识别图片中的数字

实现思路：

1. 获取训练数据集
2. 使用卷积层、池化层、全连接层构建神经网络，形成训练模型
3. 根据数据集对模型进行训练，并对正确率最高的模型进行保存
4. 使用测试集测试模型准确率，并随机抽取图片进行测试结果显示

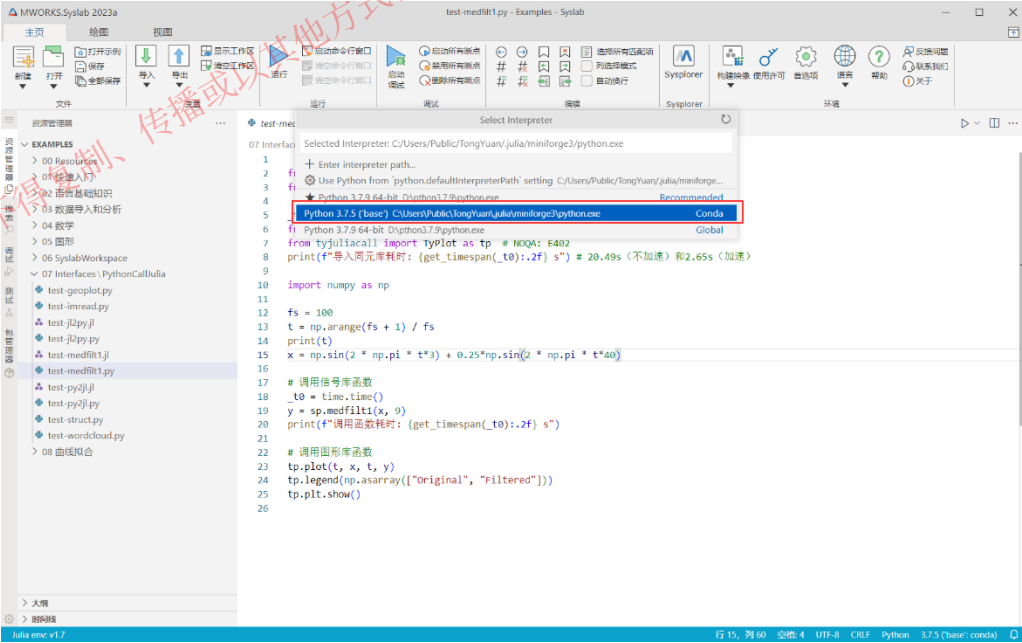
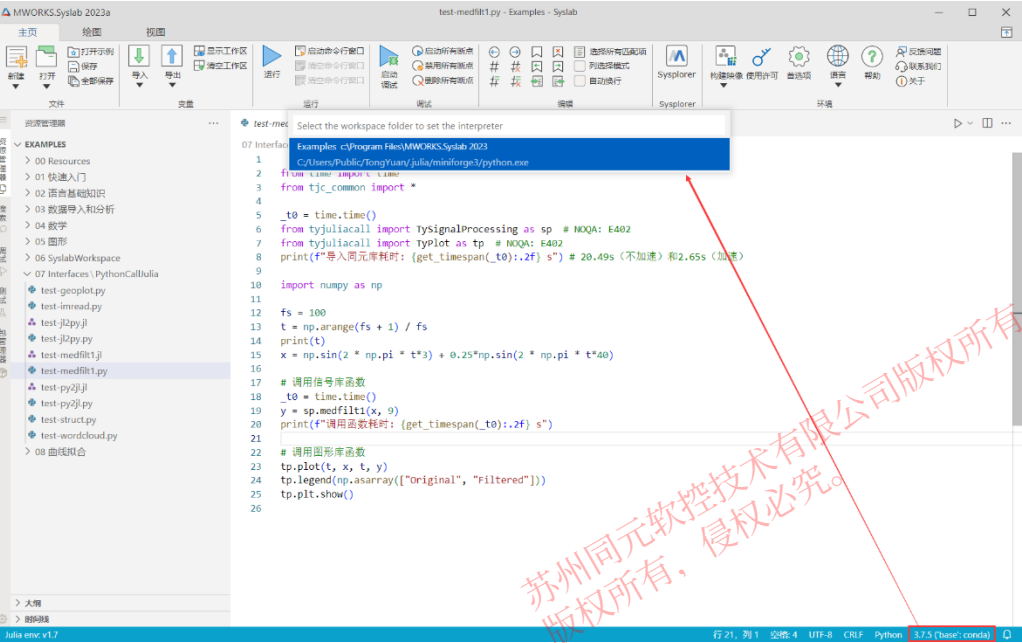
依赖函数库：

- Flux-机器学习库
- MLDatasets-通用机器学习（ML）数据集库(需要安装)
- Statistics-统计标准库
- TylImages-同元图像工具箱
- Printf-打印库
- BSON-数据结构存储库(需要安装)
- Parameters-处理数值模型参数库
- CUDA-管理GPU数据库(需要安装)



3.3 外部语言接口-Python环境

Syslab提供了一套完备的Python环境，包含同元函数库及第三方依赖包、以及Python环境及扩展包等，做到“环境隔离、开箱即用”。用户无需安装其它任何环境，在Syslab上能够实现Python编程及Python调用同元库函数。



根据文件格式启动不同的REPL



3.3 外部语言接口-Julia调用Python

在数值计算领域，存在很多用Python写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

调用Python库函数

通过 `pyimport` 和 `@pyimport` 调用Python库中函数

```
using PyCall
using TyPlot

#case 1
math = pyimport("math")
v = math.sin(pi/2)
println("v = $v")
# v = 1.0

#case 2
@pyimport numpy as np
x = np.linspace(0, 2π, 1000)
y = np.sin(x)
plot(x,y)
```

调用Python代码

通过 `py"..."` 和 `py"""..."""`，可以直接调用 Python 代码

```
module MyModule
using PyCall
function __init__()
    py"""
    def hello(s):
        return "Hello, " + s
    """
end

# 封装Python函数
hello(s) = py"hello"(s)
end

MyModule.hello("Syslab") # "Hello, Syslab"
```

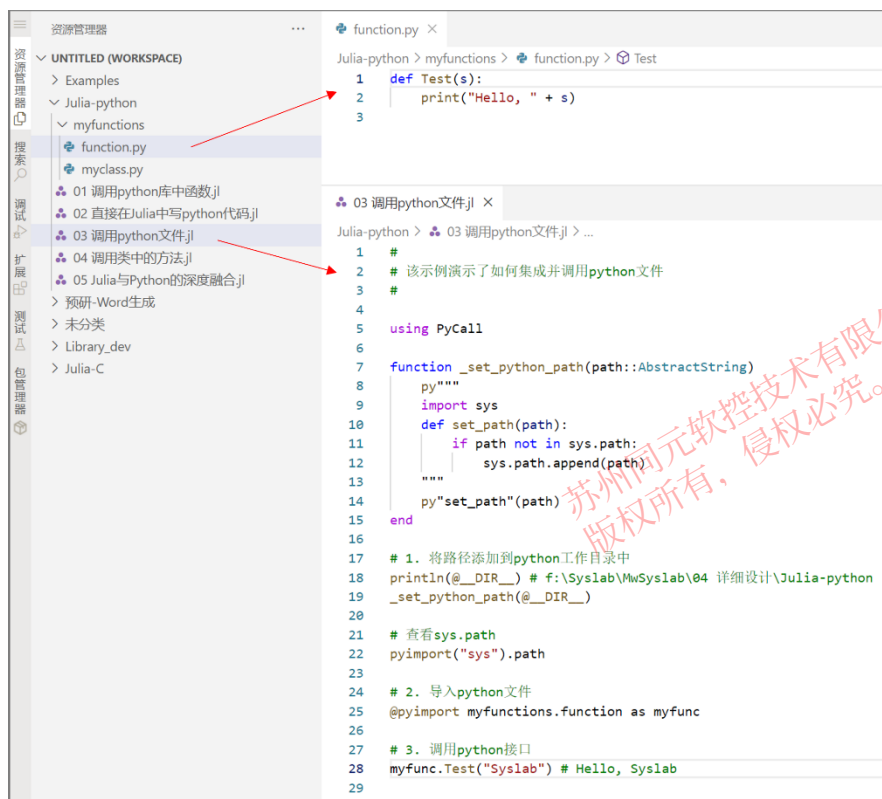


3.3 外部语言接口-Julia调用Python

在数值计算领域，存在很多用Python写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

调用Python文件

通过 `pyimport` 和 `@pyimport` 调用Python库中函数



调用Python代码

通过 `@pydef` 来创建一个python类，其方法是用julia实现的。

```
using PyCall
# python代码
py"""
import numpy.polynomial
class Doubler(numpy.polynomial.Polynomial):
    def __init__(self, x=10):
        self.x = x
    def my_method(self, arg1): return arg1 + 20
    @property
    def x2(self): return self.x * 2
    @x2.setter
    def x2(self, new_val):
        self.x = new_val / 2
print(Doubler().x2) # 20
"""
d = Doubler()
println(d.x2) # 20
d.x2 = 10
println(d.x) # 5
d.x = 15
println(d.x2) # 30
```



3.3 外部语言接口-Julia调用C/C++

在数值计算领域，存在很多用C或Fortran写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

调用c标准库函数

方式1:

```
ccall((function_name, library), returntype, (argtype1, ...), argvalue1, ...)
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)
ccall(function_pointer, returntype, (argtype1, ...), argvalue1, ...)
```

方式2:

```
@ccall library.function_name(argvalue1::argtype1, ...)::returntype
@ccall function_name(argvalue1::argtype1, ...)::returntype
@ccall $function_pointer(argvalue1::argtype1, ...)::returntype
```

调用自定义C++动态库

定义求和函数

```
//求和
extern "C" __declspec(dllexport) double GetSum(double x, double y);
```

定义求和函数

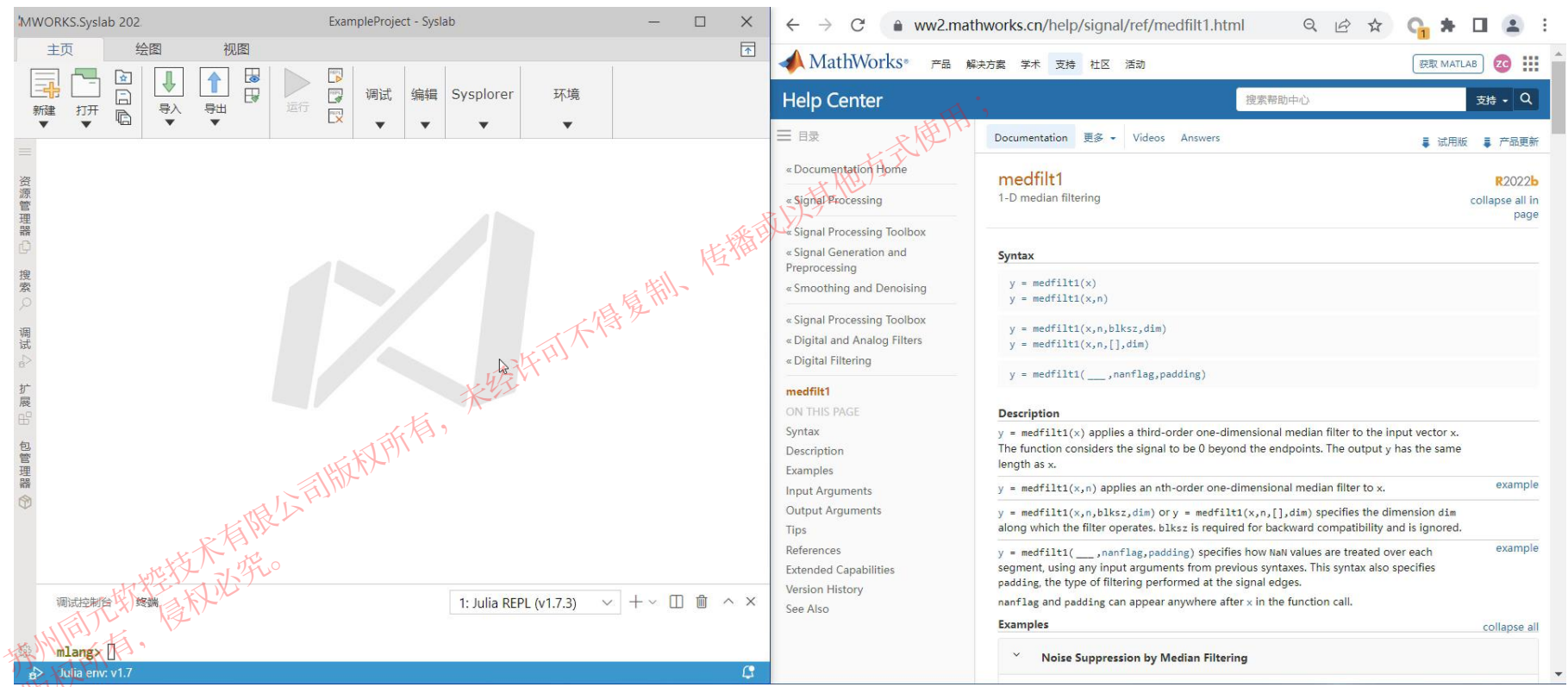
```
julia> lib = "f:\\Syslab\\Julia-
C\\ArrayMaker\\x64\\Release\\ArrayMaker" #动态库路径

julia> c = @ccall lib.GetSum(2::Cdouble, 3::Cdouble)::Cdouble
5.0
```



3.3 外部语言接口-Julia调用MATLAB

- 支持M核心语言解析
- 提供260+基础内置函数
- 支持M语言与Julia互调用
- 支持用户自定义扩展M函数



详见：Julia中文社区 2022冬季会议《赵王宏楦 — TyMLang.jl：将MATLAB代码导入Julia生态》
网址：
https://www.bilibili.com/video/BV1744y1Z7un/?spm_id_from=333.999.0.0&vd_source=8a8d166f7c4f9f90baa08d9798e00288

3.4 信息物理系统融合-与系统建模的融合

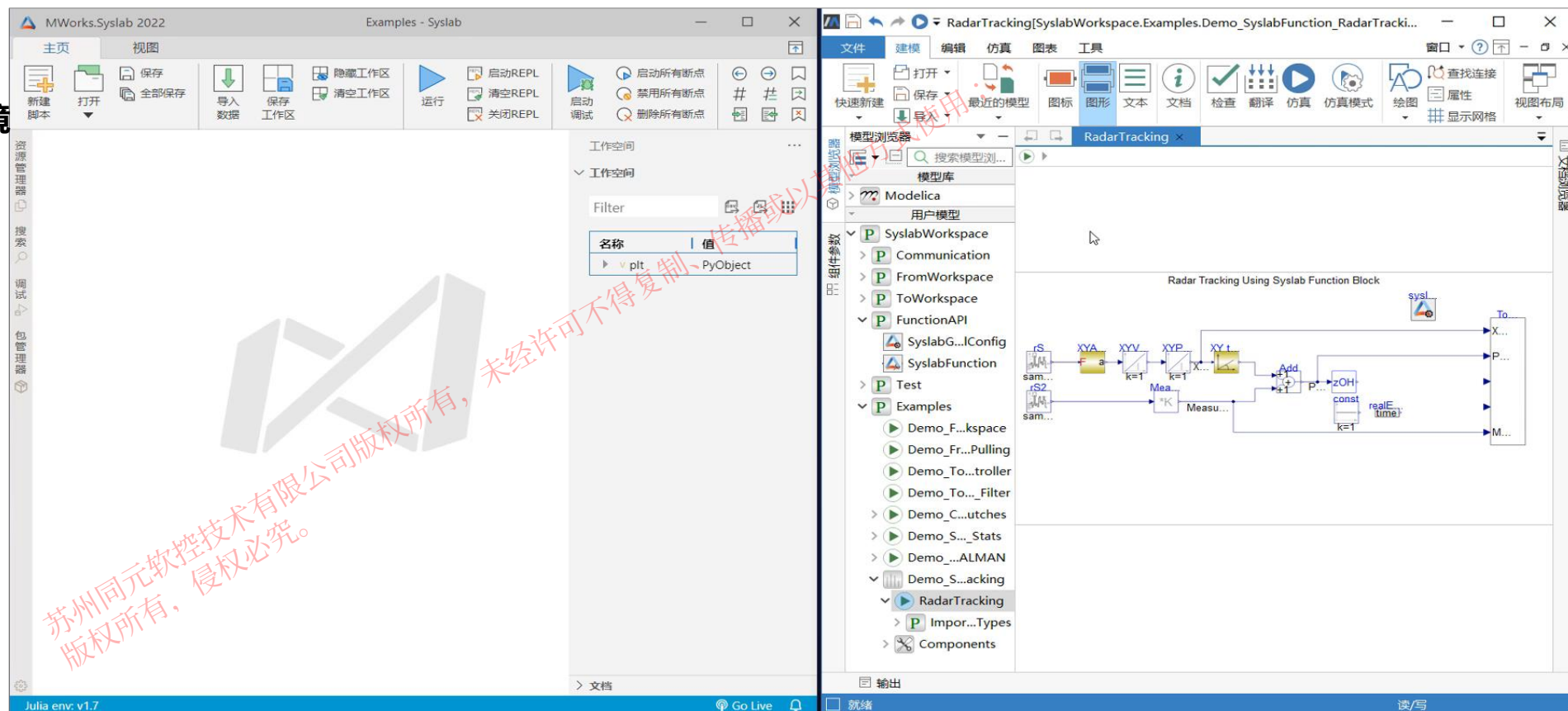
科学计算与与建模仿真一体化集成环境

□ 数据融合

- From/To Workspace

□ 接口融合

- **Syslab Function**
- Sysplorer API

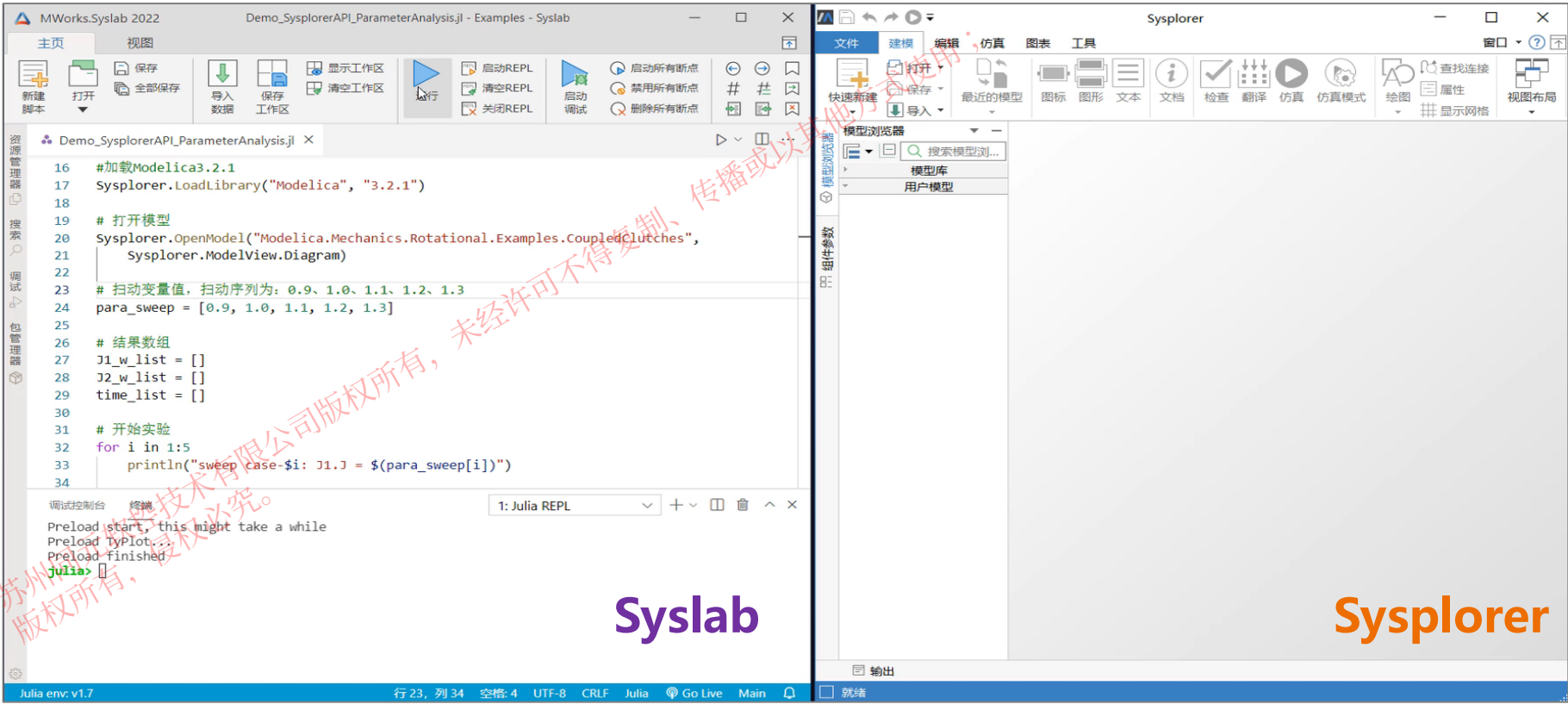


3.4 信息物理系统融合-与系统建模的融合

在Syslab科学计算环境中，通过调用Sysplorer API，实现对模型的批量仿真与数据分析

科学计算与与建模仿真一体化集成环境

- 数据融合
 - From/To Workspace
- 接口融合
 - Syslab Function
 - Sysplorer API**



Syslab

Sysplorer



目录

1. Julia语言发展历史及特点

2. Julia语言生态

3. MWORKS.Syslab功能

4. Julia语法与其他语言区别

5. 如何构建高效的Julia代码

4.1 Julia与其他语言对比

1.数组声明及操作

功能	Julia	M语言	Python
创建1维数组	A=[1;2;3]或 A=[1,2,3]	无法创建	A=np.array[1,2,3]
创建行、列矩阵	行: A=[1 2 3] 列: A=[1 2 3]'	行: A=[1 2 3] 列: A=[1;2;3]	行: A=np.array[1 2 3].reshape(1,3) 列: A=np.array[1 2 3].reshape(3,1)
k个点的线性间隔向量	A=range (1,5,length=k)	A=linspace(1,5,k)	A=np.linspace(1,5,k)
数组、字符串等的索引	从1开始; 不支持负数索引; 索引必须写全, 例如a[2:end]	从1开始; 不支持负数索引; 索引必须写全, 例如a[2:end]	从0开始; 支持负数索引; 索引可不写全, 例如a[1:]



4.1 Julia与其他语言对比

1.数组声明及操作

功能	Julia	M语言	Python
对角矩阵	<code>A=Diagonal([1,2,3])</code>	<code>A=diag([1 2 3])</code>	<code>A=np.diag([1,2,3])</code>
稀疏矩阵	<code>using SparseArrays A = spzeros(2, 2) A[1, 2] = 4 A[2, 2] = 1</code>	<code>A = sparse(2, 2) A(1, 2) = 4 A(2, 2) = 1</code>	<code>from scipy.sparse import coo_matrix A = coo_matrix(([4, 1], ([0, 1], [1, 1])), shape=(2, 2))</code>
三对角矩阵	<code>x = [1, 2, 3] y = [4, 5, 6, 7] z = [8, 9, 10] Tridiagonal(x, y, z)</code>	<code>A = [1 2 3 NaN; 4 5 6 7; NaN 8 9 0] spdiags(A',[-1 0 1], 4, 4)</code>	<code>import sp.sparse as sp diagonals = [[4, 5, 6, 7], [1, 2, 3], [8, 9, 10]] sp.diags(diagonals, [0, -1, 2]).toarray()</code>
将矩阵转换为矢量	<code>A[:]</code>	<code>A(:)</code>	<code>A=A.flatten()</code>
矩阵翻转	上/下翻转: <code>reverse(A,dims=1)</code> 左/右翻转: <code>reverse(A,dims=2)</code>	上/下翻转: <code>flipud(A)</code> 左/右翻转: <code>fliplr(A)</code>	上/下翻转: <code>np.flipud(A)</code> 左/右翻转: <code>np.fliplr(A)</code>



4.1 Julia与其他语言对比

1.数组声明及操作

功能	Julia	M语言	Python
变量赋值	数组在分配给另一个变量时不会被复制。 在A = B之后, 改变B的元素也会改变A的元素。	数组在分配给另一个变量时会被复制。 在A = B之后, 改变B的元素, A的元素值不变。	/
数组增长	不会在赋值语句中自动增长数组, 使用push!和append!来增长	a(4) = 3.2 可以创建数组 a = [0 0 0 3.2], 而 a(5) = 7 可以将它增长为 a = [0 0 0 3.2 7]	/
提取元胞数组的所有元素	[a...]	vertcat(A{:})	/
矩阵乘法	A*B代表矩阵相乘, A.*B代表逐元素相乘	A*B代表矩阵相乘, A.*B代表逐元素相乘	A*B代表逐元素相乘, A@B代表矩阵相乘



4.1 Julia与其他语言对比

2.编程操作

功能	Julia	M语言	Python
包使用	using 包	/	import numpy as np
注释一行	#This is a comment	%This is a comment	#This is a comment
注释块	<pre>#= Comment block =#</pre>	<pre>%{ Comment block %}</pre>	<pre># Block # comment # following PEP8</pre>
打印文本和变量	<pre>x = 10 println("x = \$x")</pre>	<pre>x = 10 fprintf('x = %d \n', x)</pre>	<pre>x = 10 print(f'x = {x}')</pre>
函数编写	<pre>function f(x) return x^2 end 或 f(x) = x^2 #简写形式</pre>	<pre>function out = f(x) out = x^2 end</pre>	<pre>def f(x): return x**2</pre>



4.1 Julia与其他语言对比

2.编程操作

功能	Julia	M语言	Python
函数调用	$y1,y2=f(x1,x2)$ 默认参数重新求值	$[y1,y2]=f(x1,x2)$ /	$[y1,y2]=f([x1,x2])$ 只在函数定义是对默认参数求一次值
虚数单位sqrt (-1)	im	i或者j	j
%含义	余数运算符	注释	模运算符
指数运算	符号^表示 a^b^c 认为是 $a^{(b^c)}$	符号^表示 a^b^c 认为是 $(a^b)^c$	符号**
adjoint 函数	共轭转置	提供了经典伴随，余子式的转置	/



4.1 Julia与其他语言对比

2.编程操作

功能	Julia	M语言	Python
字符串拼接	*	/	+
范围语法	x[start:step:stop]	x[start:step:stop]	x[start:(stop+1):step]
代码延续	不完整的表达式会自动延续到下一行	...	"\" 或 () 或使用块注释字符串
传参	默认按位置传参，必须使用 ";" 关键字来传递关键字参数	函数句柄	按位置传递



4.1 Julia与其他语言对比

3.控制流定义

功能	Julia	M语言	Python
if语句	<pre>if i <= N # do something end</pre>	<pre>if i <= N % do something end</pre>	<pre>if i <= N: # do something</pre>
	<pre>if i <= N # do something else # do something else end</pre>	<pre>if i <= N % do something else % do something else end</pre>	<pre>if i <= N: # do something else: # so something else</pre>
for语句	<pre>for i in 1:N # do something end</pre>	<pre>for i = 1:N % do something end</pre>	<pre>for i in range(n): # do something</pre>
while语句	<pre>while i <= N # do something end</pre>	<pre>while i <= N % do something end</pre>	<pre>while i <= N: # do something</pre>



4.2 学习材料

具体Julia语法学习可见：《Julia中文文档》
网址：<https://docs.juliacn.com/latest/>



目录

1. Julia语言发展历史及特点

2. Julia语言生态

3. MWORKS.Syslab功能

4. Julia语法与其他语言区别

5. 如何构建高效的Julia代码

5 如何构建高效的Julia代码

Julia

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> @btime mysum($(rand(1024, 1024)));
130.620 μs (0 allocations: 0 bytes)
```

MATLAB

```
1 A = rand(1024, 1024);
2 f = @() mysum(A);
3 timeit(f)
4
5 function rst = mysum(A)
6     rst = 0.0;
7     for i = 1:numel(A)
8         rst = rst + A(i);
9     end
10    return
11 end
```

1.7 ms

Python

```
In [8]: import numpy as np

In [9]: def mysum(A):
...:     rst = 0.0
...:     for i in range(A.shape[0]):
...:         for j in range(A.shape[1]):
...:             rst += A[i, j]
...:     return rst

In [10]: A = np.random.rand(1024, 1024)

In [11]: %timeit mysum(A)
161 ms ± 3.75 ms per loop (mean ± std. dev.)
```

Julia + LoopVectorization

```
julia> using LoopVectorization

julia> function mysum(A)
    rst = zero(eltype(A))
    @tturbo for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> @btime mysum($(rand(1024, 1024)));
30.425 μs (0 allocations: 0 bytes)
```

```
>> A = rand(1024, 1024);
>> f = @() sum(A);
>> timeit(f)
```

ans =

7.5822e-05

多线程 (N=8) 75 μs

Python Numpy

```
In [1]: import numpy as np

In [2]: A = np.random.rand(1024, 1024)

In [3]: %timeit A.sum()
322 μs ± 6.45 μs per loop (mean ± std. dev. of 7)
```

Julia上限很高!!!



5 如何构建高效的Julia代码



```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
131.004 μs (0 allocations: 0 bytes)
```



```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
1.869 ms (0 allocations: 0 bytes)
```



```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> f() = rand{Bool} ? 0 : 0.5
f (generic function with 1 method)

julia> A = [f() for i in 1:1024, j in 1:1024];

julia> @btime mysum($A);
25.179 ms (1048573 allocations: 16.00 MiB)
```



```
julia> function mysum(A)
    rst = 0
    for i in 1:size(A, 1)
        for j in 1:size(A, 2)
            rst += A[i, j]
        end
    end
    return rst
end
mysum (generic function with 1 method)

julia> f() = rand{Bool} ? 0 : 0.5
f (generic function with 1 method)

julia> A = [f() for i in 1:1024, j in 1:1024];

julia> @btime mysum($A);
39.142 ms (1048576 allocations: 16.00 MiB)
```

大多数Julia初学者的代码

Julia下限也很低!!!

5.1 类型稳定

Julia 的一切性能优化的可能性都建立在“**类型稳定**”的基础上

使用 @code_warntype 检查类型不稳定

```
julia> function mysum(A)
    rst = 0
    for i in eachindex(A)
        rst += A[i] ::Union{Int,eltype(A)}
    end
    return rst
end
```

```
julia> @code_warntype mysum(rand(4, 4))
MethodInstance for mysum(::Matrix{Float64})
  from mysum(A; dims) in Main at REPL[27]:1
Arguments
  #self#::Core.Const(mysum)
  A::Matrix{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Union{Float64, Int64}
  i::Int64
Body::Union{Float64, Int64}
1 --      (rst = 0)
   %2  = Main.eachindex(A)::Base.OneTo{Int64}
   (@_3 = Base.iterate(%2))
   %4  = (@_3 === nothing)::Bool
   %5  = Base.not_int(%4)::Bool
   goto #4 if not %5
2 --- %7  = @_3::Tuple{Int64, Int64}
   (i = Core.getfield(%7, 1))
   %9  = Core.getfield(%7, 2)::Int64
   %10 = rst::Union{Float64, Int64}
   %11 = Base.getindex(A, i)::Float64
   (rst = %10 + %11)
   (@_3 = Base.iterate(%2, %9))
   %14 = (@_3 === nothing)::Bool
   %15 = Base.not_int(%14)::Bool
   goto #4 if not %15
3 --      goto #2
4 ---      return rst
```

苏州同元软控技术有限公司版权所有，未经许可，不得复制或传播。
版权所有，侵权必究。

5.1 类型稳定

Julia 的一切性能优化的可能性都建立在“**类型稳定**”的基础上

1. 函数返回值的类型是不变的



```
julia> f() = rand{Bool} ? 0 : 1.0
f (generic function with 1 method)

julia> @code_warntype f()
MethodInstance for f()
  from f() in Main at REPL[6]:1
Arguments
  #self#::Core.Const(f)
Body::Union{Float64, Int64}
1 — %1 = Main.rand(Main.Bool)::Bool
   └─ goto #3 if not %1
2 — return 0
3 — return 1.0

julia> @btime f();
3.910 ns (0 allocations: 0 bytes)

julia> @btime sum($([f() for i in 1:1024]));
21.852 μs (1023 allocations: 15.98 KiB)
```



```
julia> f() = rand{Bool} ? 0.0 : 1.0
f (generic function with 1 method)

julia> @code_warntype f()
MethodInstance for f()
  from f() in Main at REPL[10]:1
Arguments
  #self#::Core.Const(f)
Body::Float64
1 — %1 = Main.rand(Main.Bool)::Bool
   └─ goto #3 if not %1
2 — return 0.0
3 — return 1.0

julia> @btime f();
6.828 ns (0 allocations: 0 bytes)

julia> @btime sum($([f() for i in 1:1024]));
69.255 ns (0 allocations: 0 bytes)
```


5.1 类型稳定

Julia 的一切性能优化的可能性都建立在 “类型稳定”的基础上

2. 函数的返回值类型可以根据函数输入的类型唯一确定



```
julia> function f(x)
    if x < 0.5
        return Int(0)
    else
        return Float64(1.0)
    end
end

f (generic function with 2 methods)

julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[16]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Union{Float64, Int64}
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.Int(0)::Core.Const(0)
└─ return %3
3 - %5 = Main.Float64(1.0)::Core.Const(1.0)
└─ return %5

julia> @btime sum($([f(rand()) for i in 1:1024]));
22.534 μs (1023 allocations: 15.98 KiB)
```



```
julia> function f(x)
    if x < 0.5
        return zero(x)
    else
        return one(x)
    end
end

f (generic function with 2 methods)

julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0.0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1.0)
└─ return %5

julia> @btime sum($([f(rand()) for i in 1:1024]));
70.824 ns (0 allocations: 0 bytes)
```

```
julia> @code_warntype f(1)
MethodInstance for f(::Int64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Int64
Body::Int64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1)
└─ return %5

julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0.0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1.0)
└─ return %5
```

5.1 类型稳定

Julia 的一切性能优化的可能性都建立在 “类型稳定”的基础上

3. 函数的底层类型也需要稳定

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
```

mysum (generic function with 1 method)

```
julia> gen_data(f, n::Int) = [f() for i in 1:n, j in 1:n]
```

gen_data (generic function with 1 method)

```
julia> f_slow() = rand{Bool} ? 0 : 1.0
```

f_slow (generic function with 1 method)

```
julia> h(n) = mysum(gen_data(f_slow, 1024))
```

h (generic function with 1 method)

```
julia> @btime h(1024);
```

35.263 ms (1571575 allocations: 39.98 MiB)

```
julia> f_fast() = rand{Bool} ? 0.0 : 1.0
```

f_fast (generic function with 1 method)

```
julia> h(n) = mysum(gen_data(f_fast, 1024))
```

h (generic function with 1 method)

```
julia> @btime h(1024);
```

5.317 ms (2 allocations: 8.00 MiB)

未经许可不得复制、传播或以其他方式使用；

5.2 具体数值类型

具体数值类型意味着编译器知道如何去优化它



```
julia> function gen_data(n)
    A = []
    for i in 1:n
        push!(A, i)
    end
    return A
end
gen_data (generic function with 2 methods)

julia> @btime sum(gen_data(1024));
30.152 μs (1512 allocations: 45.38 KiB)
```



```
julia> function gen_data(n)
    A = Int[]
    for i in 1:n
        push!(A, i)
    end
    return A
end
gen_data (generic function with 2 methods)

julia> @btime sum(gen_data(1024));
5.887 μs (6 allocations: 21.86 KiB)
```

```
julia> [1, 2, 3]
3-element Vector{Int64}:
 1
 2
 3

julia> Any[1, 2, 3]
3-element Vector{Any}:
 1
 2
 3

julia> []
Any[]
```

数据类型越具体，编译器就越有可能给出好的优化策略



5.2 具体数据类型

具体数值类型意味着编译器知道如何去优化它

```
julia> dist(x::Point2D, y::Point2D) = sqrt((x.x - y.x)^2 + (x.y - y.y)^2)
dist (generic function with 1 method)

julia> function f(points)
    p = Point2D(0.0, 0.0)
    out = Vector{Float64}(undef, length(points))
    for i in 1:length(points)
        out[i] = dist(p, points[i])
    end
    return out
end
f (generic function with 1 method)

julia> gen_points(n) = [Point2D(rand(), rand()) for i in 1:n]
gen_points (generic function with 1 method)
```



```
julia> struct Point2D
    x
    y
end

julia> points = gen_points(1024);

julia> @btime f($points);
82.900 μs (6658 allocations: 112.14 KiB)
```

如果我们让结构体的元素的类型
变为具体类型...



```
julia> struct Point2D
    x::Float64
    y::Float64
end

julia> points = gen_points(1024);

julia> @btime f($points);
1.250 μs (1 allocation: 8.12 KiB)
```

5.3 不可变结构体

优先使用不可变结构体而非可变结构体

```
julia> struct Point2D
        x::Float64
        y::Float64
    end

julia> points = gen_points(1024^2);

julia> @btime f($points);
2.384 ms (2 allocations: 8.00 MiB)
```

```
julia> mutable struct Point2D
        x::Float64
        y::Float64
    end

julia> points = gen_points(1024^2);

julia> @btime f($points);
2.961 ms (2 allocations: 8.00 MiB)
```

如果我们让结构体变为可变结构体...

5.3 不可变结构体

优先使用不可变结构体而非可变结构体

```
julia> struct Point2D
        x::Float64
        y::Float64
    end

julia> points = gen_points(1024^2);

julia> @btime f($points);
2.384 ms (2 allocations: 8.00 MiB)
```



```
julia> mutable struct Point2D
        x::Float64
        y::Float64
    end

julia> points = gen_points(1024^2);

julia> @btime f($points);
2.961 ms (2 allocations: 8.00 MiB)
```

如果我们让结构体变为可变结构体...

5.3 不可变结构体

优先使用不可变结构体而非可变结构体

```
julia> function g(points)
    p = Point2D(0.0, 0.0)
    out = Vector{Float64}(undef, length(points))
    for i in 1:length(points)
        pi = points[i]
        pi.y = 0.0
        out[i] = dist(p, pi)
    end
    return out
end
g (generic function with 1 method)

julia> points = gen_points(1024);

julia> @btime g($points);
1.582 μs (1 allocation: 8.12 KiB)
```

```
julia> function g(points)
    p = Point2D(0.0, 0.0)
    out = Vector{Float64}(undef, length(points))
    for i in 1:length(points)
        pi = Point2D(points[i].x, 0.0)
        out[i] = dist(p, pi)
    end
    return out
end
g (generic function with 1 method)

julia> points = gen_points(1024);

julia> @btime g($points);
1.544 μs (1 allocation: 8.12 KiB)
```

创建、而非修改

5.3 不可变结构体

某些情况下，不可变结构体可以提供 0 开销抽象

```
julia> A = [rand() for i in 1:1024,j in 1:1024];  
julia> @btime sum($A);  
144.664 μs (0 allocations: 0 bytes)
```



```
julia> struct MyFloat64 <: AbstractFloat  
    x::Float64  
end  
  
julia> Base.:(+)(a::MyFloat64, b::MyFloat64) = MyFloat64(a.x + b.x)  
  
julia> A = [MyFloat64(rand()) for i in 1:1024,j in 1:1024];  
  
julia> @btime sum($A);  
143.835 μs (0 allocations: 0 bytes)
```

不可变结构体



```
julia> mutable struct MyFloat64 <: AbstractFloat  
    x::Float64  
end  
  
julia> Base.:(+)(a::MyFloat64, b::MyFloat64) = MyFloat64(a.x + b.x)  
  
julia> A = [MyFloat64(rand()) for i in 1:1024,j in 1:1024];  
  
julia> @btime sum($A);  
6.937 ms (1048575 allocations: 16.00 MiB)
```

可变结构体

5.4 内存顺序与布局

在一门足够快的语言中，for 循环的内存迭代顺序会成为影响性能的主要因素



```
julia> function mysum(A, B)
    rst = 0.0
    @inbounds for i in 1:size(A, 1)
        @simd for j in 1:size(A, 2)
            rst += A[i, j] + B[i, j]
        end
    end
    return rst
end

mysum (generic function with 2 methods)

julia> A, B = rand(1024, 1024), rand(1024, 1024);

julia> @btime mysum($A, $B);
8.585 ms (0 allocations: 0 bytes)
```



```
julia> function mysum(A, B)
    rst = 0.0
    @inbounds for j in 1:size(A, 2)
        @simd for i in 1:size(A, 1)
            rst += A[i, j] + B[i, j]
        end
    end
    return rst
end

mysum (generic function with 2 methods)

julia> A, B = rand(1024, 1024), rand(1024, 1024);

julia> @btime mysum($A, $B);
413.684 μs (0 allocations: 0 bytes)
```

5.5 向量化编程

向量化编程很好，但Julia中并不是主要用法。

Python/MATLAB 中常见的向量化编程

```
function f(X, Y, Z)
    M = X .> Y
    C = similar(X)
    C[M] .= X[M] .* Z[M]
    M .= !M
    C[M] .= X[M] .+ Y[M]
    return C
end
```

Julia 中更多的是标量逻辑

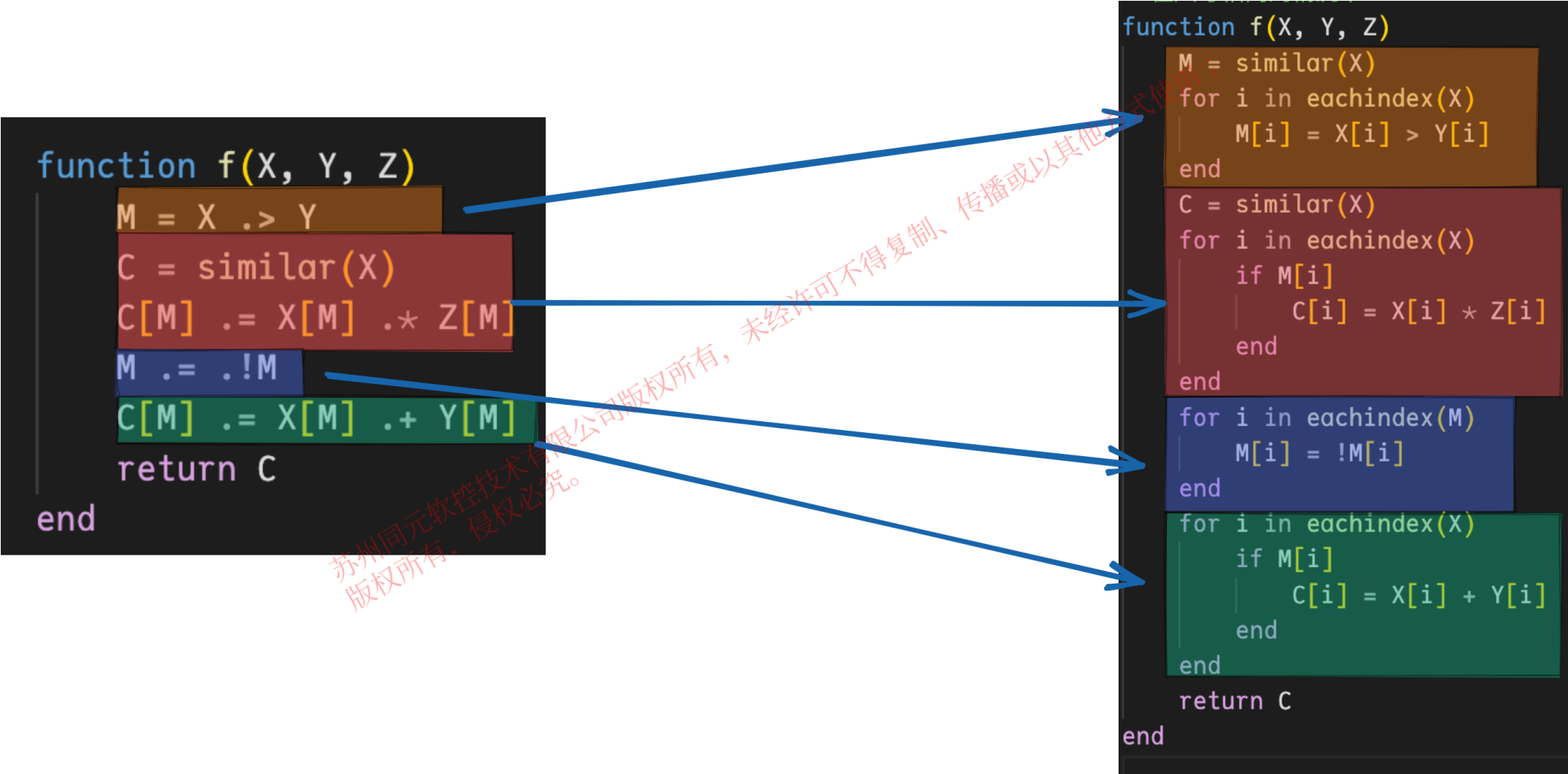
```
function g(x, y, z)
    return x > y ? x * z : x + y
end
```

在 CPU 上，向量化编程不会比普通的标量函数更快

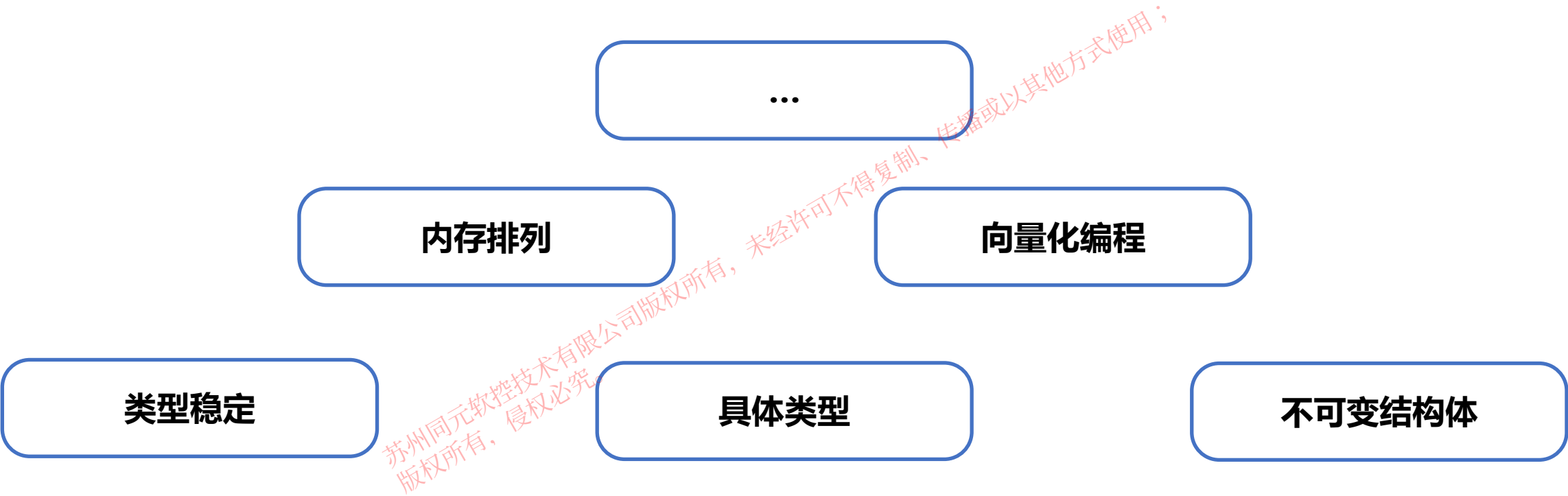
```
julia> using BenchmarkTools
julia> X, Y, Z = rand(4096), rand(4096), rand(4096);
julia> f(X, Y, Z) == g.(X, Y, Z)
true
julia> @btime f($X, $Y, $Z);
15.892 μs (14 allocations: 133.53 KiB)
julia> @btime g.($X, $Y, $Z);
1.563 μs (2 allocations: 32.05 KiB)
```

5.5 向量化编程

向量化编程背后实际发生的是：



5.6 总结



感谢您的聆听!

Thanks

苏州同元软控技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。